

Relazione taw Andrea Giazzon 896876

Architettura del Sistema

- **Docker** permette al sistema di essere progettato seguendo un approccio a container, suddiviso in tre ambienti isolati. Docker è uno strumento di sviluppo che permette di creare un'**Immagine** (una versione leggera del sistema che si vuole emulare) per poi eseguirla all'interno di un **Container**. Ogni container è un ambiente isolato che esegue l'immagine scelta seguendo le istruzioni contenute nel **Dockerfile**, il file che definisce come costruire l'ambiente di lavoro. Il vantaggio principale è che Docker non emula l'hardware (come le macchine virtuali tradizionali), ma virtualizza solo il sistema operativo. Questo ci permette di avere tre container che comunicano tra loro, dove ognuno installa solo l'ambiente e gli strumenti necessari per far funzionare un singolo pezzo della nostra applicazione. Questo isolamento delle tecnologie rende il progetto estremamente scalabile, modulare e portabile. L'intera architettura viene gestita tramite il file **docker-compose.yml**, che ha il compito di orchestrare e collegare i tre container attraverso una rete virtuale interna dedicata. L'architettura prevede tre container distinti:
 1. **MongoDB Container:** Dedicato alla persistenza dei dati. Per questo servizio, Docker utilizza i **Volumi**: una cartella reale del PC viene "montata" all'interno del container, garantendo che le informazioni salvate (come voli e utenti) sopravvivano al riavvio o alla distruzione del container stesso.
 2. **Backend Container:** Ospita l'ambiente Node.js ed Express, gestendo la logica del server.
 3. **Frontend Container:** Ospita l'applicazione Angular e l'interfaccia utente.
- **Data Layer (MongoDB)** Gestisce la persistenza dei dati tramite un database **NoSQL** orientato ai documenti. A differenza dei database relazionali (SQL) che frammentano i dati in tabelle, righe e colonne, MongoDB organizza le informazioni in **Collezioni** (insiemi di dati omogenei) e **Documenti**. Ogni documento è una struttura dati complessa (in formato BSON, simile al JSON) capace di ospitare al suo interno oggetti annidati e array. Questo permette di aggregare tutte le informazioni correlate in un'unica entità (ad esempio, un singolo documento "Volo" contiene già i prezzi e la configurazione dei posti), eliminando la necessità di eseguire costose operazioni di *JOIN* per ricostruire i dati. Inoltre, questa struttura garantisce un'elevata flessibilità: i documenti all'interno della stessa collezione non sono vincolati a uno schema rigido e possono avere campi differenti, permettendo al database di adattarsi dinamicamente all'evoluzione del progetto.
- **Backend Service (Node.js & Express)** Rappresenta il core logico dell'applicazione, sviluppato in ambiente **Node.js**, che ci permette di eseguire codice JavaScript direttamente sul server. La sua caratteristica principale è l'architettura **single-threaded non bloccante**: il server non si ferma in attesa delle operazioni di I/O (come le query al

database), ma invia la richiesta e passa immediatamente a servire quella successiva. Solo quando il database risponde, viene scatenato un evento che riporta il risultato nel flusso principale. Tutto questo è reso possibile dall'**Event Loop**, un ciclo infinito che attende il verificarsi di eventi per eseguire le funzioni di callback associate.

- **Framework Express** Semplifica la struttura del server gestendo il **routing**, ovvero indirizzando le richieste HTTP verso gli specifici endpoint destinatari. Consente inoltre l'implementazione di **middleware**, funzioni intermedie che elaborano la richiesta prima che arrivi a destinazione. Nel progetto, questo meccanismo è stato utilizzato per la gestione del **JWT**: un middleware intercetta ogni chiamata protetta, ne verifica la validità del token e solo in caso di successo la inoltra all'endpoint finale, gestendo automaticamente anche la compilazione corretta degli **HTTP headers**.
 - **API RESTful**: Espone gli endpoint sulla porta **3005** per la gestione delle risorse (spiegato nel dettaglio in seguito)
 - **ODM (Object Data Modeling)** Utilizziamo **Mongoose**, una libreria ODM per Node.js e MongoDB, per la modellazione dei dati e l'interazione strutturata. L'ODM agisce da "ponte" con il codice JavaScript del server, permettendoci di definire degli **Schemi**. Questo passaggio è fondamentale perché, nativamente, una collezione MongoDB non ha filtri e accetterebbe qualsiasi dato; grazie a Mongoose, invece, possiamo gestire controlli e validazioni direttamente nel codice (come implementato nel progetto). Inoltre, l'ODM ci permette di utilizzare metodi JavaScript intuitivi per la ricerca (come `.find()`) e la funzione `.populate()`, che serve a simulare le relazioni tra documenti (in modo simile alle JOIN dei database relazionali).
 - **Sicurezza**: Implementa un sistema di autenticazione stateless basato su **JWT** (JSON Web Token).
 - **Real-Time**: Gestisce la comunicazione bidirezionale tramite **WebSocket** (Socket.io) un protocollo di comunicazione che permette uno scambio di dati **bidirezionale e continuo** tra un client (il tuo frontend Angular) e un server (il tuo backend Node.js).
- **Frontend Client (Angular)** L'interfaccia utente è una **Single Page Application (SPA)** sviluppata con Angular. Questa architettura migliora l'esperienza utente poiché non richiede il ricaricamento completo della pagina a ogni navigazione, ma aggiorna dinamicamente solo le porzioni necessarie del DOM. Il sistema è modulare e basato su **Componenti**, che incapsulano la struttura (HTML), la logica (TypeScript) e lo stile di ogni singola vista. La logica e la comunicazione dati è invece delegata ai **Services**, separandola dalla vista. Per la gestione delle operazioni asincrone, utilizziamo il pattern degli **Observables** (tramite libreria RxJS), che ci permette di rimanere in attesa della risposta del server e reagire istantaneamente non appena i dati vengono ricevuti.
- **UX/UI (Tailwind CSS)** Lo stile grafico e il layout sono realizzati tramite **Tailwind CSS**, garantendo un design moderno e leggero. L'adozione di un approccio **Utility-First** ci permette di utilizzare classi di basso livello che mappano direttamente le proprietà CSS. Questo consente di definire lo stile direttamente all'interno dell'HTML, ottenendo un controllo totale sull'interfaccia e una sintassi standardizzata, senza la necessità di scrivere fogli di stile esterni complessi.

- **Integrazione** : Quando l'utente interagisce con l'interfaccia **Angular**, i **Services** del frontend inviano chiamate HTTP asincrone al **Backend**, che a sua volta interroga **MongoDB** tramite **Mongoose**. I dati tornano indietro come oggetti JSON, pronti per essere mostrati dinamicamente nella SPA senza ricaricare la pagina. Per le operazioni che richiedono reattività immediata, come la prenotazione dei posti, il sistema passa alla comunicazione bidirezionale. Tramite **Socket.io**, il Frontend e il Backend restano "ascolto" l'uno dell'altro. Tutto questo avviene dentro una **rete virtuale dedicata** creata da Docker Compose. Il

Data Model

User

```
const mongoose = require('mongoose');
const crypto = require("crypto");

const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true
  },

  role: {
    type: String,
    enum: ['admin', 'passenger', 'airline'],
    default: 'passenger',
    required: true
  },

  //dati passeggero
  name : {
    type: String,
    required: function (){
      return this.role === 'passenger';
    }
  },

  surname : {
    type: String,
    required: function (){
      return this.role === 'passenger';
    }
  },

  },
```

```
birthdate : {
  type: Date,
  required: function () {
    return this.role === 'passenger';
  }
},

phonenumber : {
  type: String,
  required: function () {
    return this.role === 'passenger';
  }
},

paymentAddress : {

  type: String,
  required: function () {
    return this.role === 'passenger';
  }
},

company : {
  type: String,
  unique: true,
  sparse: true, // permette valori nulli multipli
  required: function () {
    return this.role === 'airline';
  }
},

dateofcreation: {
  type: Date,
  default: Date.now
},

salt: {
  type: String,
  required: false
},

digest: {
  type: String,
```

```

    required: false
  }

});

userSchema.methods.setPassword = function (password) {
  this.salt = crypto.randomBytes(16).toString('hex');

  const hmac = crypto.createHmac('sha512', this.salt);

  hmac.update(password);
  //metto password criptata nel diges
  this.digest = hmac.digest('hex');

};

userSchema.methods.validatePassword = function (password) {
  const hmac = crypto.createHmac('sha512', this.salt);
  hmac.update(password);
  const digest = hmac.digest('hex');
  return (this.digest === digest);
}

const User = mongoose.model('User', userSchema);

module.exports = User;

```

- **Campi Comuni:** per tutti gli utenti
 - `_id` : ObjectId (generato automaticamente).
 - `email` : Stringa (Univoca, Obbligatoria).
 - `role` : Stringa (Enum: 'admin' , 'passenger' , 'airline').
 - `dateofcreation` : Data (Default: oggi).
 - `salt` : Stringa (per la crittografia password).
 - `digest` : Stringa (hash della password).
- **Campi specifici per Passeggeri (`role`: 'passenger'):**
 - `name` : Stringa.
 - `surname` : Stringa.
 - `birthdate` : Data.
 - `phonenummer` : Stringa.
 - `paymentAddress` : Stringa.
- **Campi specifici per Compagnie Aeree (`role`: 'airline'):**
 - `company` : Stringa (Univoca).

Per gestire lo schema `User`, ho deciso di unificare i dati di tutti i tipi di account in un unico modello. I campi non pertinenti al ruolo specifico rimarranno semplicemente vuoti o non definiti. Oltre al campo `_id`, generato univocamente da MongoDB, ho imposto vincoli di unicità (`unique: true`) sia sulla **email** che sul **nome della compagnia**. Questo garantisce l'integrità del database, evitando che due utenti condividano la stessa email o che due compagnie aeree abbiano lo stesso nome, prevenendo ambiguità.

All'interno dello schema sono state implementate due funzioni per la sicurezza, basate sugli esempi del corso:

1. **Gestione Password:** Vengono utilizzati i campi `salt` (stringa casuale generata dal server) e `digest`.
2. **Hashing:** La password non viene salvata in chiaro; viene criptata combinando il `salt` e la password effettiva tramite `hmac.update(password)`.

Infine, l'uso delle funzioni nel campo `required` assicura la validazione condizionale: specifici campi sono obbligatori solo se l'utente ricopre un determinato ruolo.

Aeroporti

```
const mongoose = require('mongoose');

const airportSchema = new mongoose.Schema({
  code : { //tipo ICE VCE BLN TRV
    type: String,
    required: true,
    unique: true,
    uppercase: true,
    minLength: 3,
    maxLength: 3
  },

  name : { // nome nome completo
    type: String,
    required: true
  },

  city : {
    type: String,
    required: true
  },

  country : {
    type: String,
    required: true
  },

  active: {
```

```

        type: Boolean,
        default: true
    }

});

module.exports = mongoose.model('Airport', airportSchema);

```

Questo schema raccoglie tutti i dati identificativi dell'aeroporto. È stato introdotto il campo `active` per gestire la **cancellazione logica**: in caso di rimozione, l'aeroporto non viene eliminato fisicamente dal database (operazione che corromperebbe i riferimenti in vecchi biglietti o prenotazioni), ma viene settato come inattivo (`active: false`), rendendolo non selezionabile per nuovi voli ma mantenendo lo storico.

- `_id` : ObjectId.
- `code` : Stringa (Codice IATA es. "FCO", Univoco, 3 caratteri maiuscoli).
- `name` : Stringa (Nome completo dell'aeroporto).
- `city` : Stringa (Città).
- `country` : Stringa (Nazione).
- `active` : Booleano (Stato operativo, default `true`).

Airplanes

```

const mongoose = require('mongoose');

const seats = new mongoose.Schema({
  rows: { type: Number, default: 0 },
  seatsPerRow: { type: Number, default: 0 },
  seatLetters: {
    type: String,
    uppercase: true,
    trim: true,
    match: /^[A-Z]+$/,
    default: "",
    validate : {
      validator: function(v) {
        if (!v) return true;
        return new Set(v).size === v.length;
      },
      message: props => `Lettere dei posti duplicate nella sezione:
${props.value}`
    }
  },
  numberOfSeats: { type: Number }
});

const airplaneSchema = new mongoose.Schema({

```

```

    airplaneModel : {
      type: String,
      required: true,
      unique: true
    },
    capacity : {
      economy : seats,
      business : seats,
      firstclass : seats
    },
    totalSeats: {
      type: Number,
      default: 0
    },
    active: {
      type: Boolean,
      default: true
    }
  });

//controllo posti e controllo dei dati inseriti
airplaneSchema.pre('save', function(next) {
  const totalSeatsPerClass = (seats) => {
    if (!seats || seats.rows === 0) {
      seats.numberOfSeats = 0;
      return 0;
    }

    if(seats.seatsPerRow !== seats.seatLetters.length){
      throw new Error(`Errore Configurazione Aereo: Nella sezione ci
sono
      ${seats.seatsPerRow} posti dichiarati ma le lettere sono
      "${seats.seatLetters}"`);
    }else{
      return seats.numberOfSeats = seats.rows *
seats.seatLetters.length;
    }
  };
  let total =0;
  try{
    total += totalSeatsPerClass(this.capacity.economy, 'Economy');
    total += totalSeatsPerClass(this.capacity.business, 'Business');
    total += totalSeatsPerClass(this.capacity.firstclass, 'First
Class');
  }catch(err){
    return next(err);
  }
  this.totalSeats = total;
  next();

```

```
});  
  
module.exports = mongoose.model('Airplane', airplaneSchema);
```

La collezione `Airplanes` rappresenta i modelli fisici degli aerei. La struttura è stata progettata considerando che ogni modello può avere una diversa configurazione di sedili e capienza.

- `airplaneModel` : Stringa (Nome modello, Univoco).
- `totalSeats` : Numero (Calcolato automaticamente).
- `active` : Booleano.
- `capacity` : Oggetto contenente la configurazione per le tre classi (`economy` , `business` , `firstclass`). Ogni classe ha:
 - `rows` : Numero di file.
 - `seatsPerRow` : Posti per fila.
 - `seatLetters` : Stringa (es. "ABCDEF", deve corrispondere al numero di posti per fila).
 - `numberOfSeats` : Numero (totale posti per quella classe).

Anche qui il campo `active` segue la logica della cancellazione logica per preservare la coerenza delle prenotazioni passate. Per gestire la flessibilità della configurazione (es. una classe potrebbe avere file con sedili "A C Z" saltando alcuni posti), ho creato un sub-schema `seats` . Le lettere sono salvate esplicitamente per permettere configurazioni non standard. Sono stati implementati controlli (hooks `pre-save` e `validators`) per garantire l'integrità dei dati, calcolando automaticamente il numero totale dei posti e verificando che il numero di lettere corrisponda ai posti per fila e che non ci siano duplicati. Sebbene il frontend debba già prevenire questi errori, questi controlli backend aggiungono un livello di robustezza al sistema, ed erano stati sviluppati inizialmente quindi ho preferito lasciarli nel codice.

Routes

```
const mongoose = require('mongoose');  
  
const routeSchema = new mongoose.Schema({  
  airlineId: {  
    type: mongoose.Schema.Types.ObjectId,  
    ref: 'User',  
    required: true  
  },  
  
  departureAirport : {  
    type: mongoose.Schema.Types.ObjectId,  
    //in sostanza un riferimento ad un oggetto di un'altra collezione ma
```

```

l'oggetto intero
    ref: 'Airport',
    required: true
  },

  arrivalAirport : {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Airport',
    required: true
  },

  active: {
    type: Boolean,
    default: true
  }
});

routeSchema.index( //no salvataggio di rotte duplicate
  { airlineId: 1, departureAirport: 1, arrivalAirport: 1 },
  { unique: true }
);

routeSchema.pre('save', function(next) {
  if (this.departureAirport.equals(this.arrivalAirport)) {
    return next(new Error('gli airports di partenza e arrivo sono uguali'));
  }
  next();
});

module.exports = mongoose.model('Route', routeSchema);

```

Ogni rotta (Route) definisce un collegamento operato da una specifica compagnia aerea tra due aeroporti. Il modello utilizza riferimenti (ObjectId) agli aeroporti di partenza e arrivo, mantenendo anche qui il campo `active` per disattivare la rotta senza eliminarla. Sono presenti controlli specifici per l'integrità:

- Verifica che l'aeroporto di partenza e quello di arrivo non coincidano.
- Un indice univoco composto (`airlineId + departureAirport + arrivalAirport`) per impedire la creazione di rotte duplicate per la stessa compagnia.
- `airlineId` : ObjectId (Riferimento a `User` -> Compagnia aerea).
- `departureAirport` : ObjectId (Riferimento a `Airport`).
- `arrivalAirport` : ObjectId (Riferimento a `Airport`).
- `active` : Booleano.

Flights

```
const mongoose = require('mongoose');

const flightSchema = new mongoose.Schema({

  flightNumber: {
    //numero volo ripetibile anche in giorni diversi
    type: String,
    required: true,
  },

  company : {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },

  prices : {
    economy : {
      type: Number,
      required: true,
      min: 0
    },

    business : {
      type: Number,
      required: true,
      min: 0
    },

    firstclass : {
      type: Number,
      required: true,
      min: 0
    },

    extras: {
      baggage: {
        type: Number,
        default: 30,
        min: 0
      },

      legroom: {
        type: Number,
        default: 15,
        min: 0
      }
    }
  }
});
```

```

    },

    priorityBoarding: {
      type: Number,
      default: 10,
      min: 0
    }
  }
},

departureTime : {
  type: Date,
  required: true
},

arrivalTime : {
  type: Date,
  required: true
},

airplane :{
  type: mongoose.Schema.Types.ObjectId,
  ref: 'Airplane',
  required: true
},

route :{
  type: mongoose.Schema.Types.ObjectId,
  ref: 'Route',
  required: true
},

active: {
  type: Boolean,
  default: true
},

bookedSeats : {
  type: [{
    seat: {
      type: String,
      required: true
    },

    travelClass: {
      type: String,
      required: true,
      enum: ['economy', 'business', 'firstclass']
    },
  },
  _id: false // mongoose no id

```

```

    }],
    required: true,
    default: [],
    validate : {
        validator: function(v) {
            const seatNumbers =
            v.map(item => `${item.seat}-${item.travelClass}`);
            //array di stringhe univoche 1A-economy 1A-business
            let set = new Set(seatNumbers);
            //set non ammette duplicati
            return set.size === seatNumbers.length;
        },
        error : 'posti duplicati nel volo'
    }
  },
});

flightSchema.pre('save', function(next) {
  //controllo che l'orario di arrivo sia successivo a quello di partenza
  if(this.arrivalTime <= this.departureTime)
    {throw new Error
      (`errore orario di partenza precedente o uguale a quello di
      arrivo`);
    }
  next();
});

module.exports = mongoose.model('Flight', flightSchema);

```

Lo schema `Flight` costituisce il punto nevralgico del database, in quanto rappresenta l'istanza concreta di un volo. Esso aggrega riferimenti multipli agli altri schemi: **Aeroporto** (implicito nella Rotta), **Aereo**, **Compagnia Aerea** e **Rotta**.

Il `flightNumber` è il codice identificativo del volo; non è univoco in senso assoluto poiché la compagnia può operare lo stesso numero di volo in giorni diversi (l'unicità del documento è garantita dall' `_id`). Ogni documento contiene inoltre gli orari di partenza e arrivo e la struttura dei prezzi (base per classe e costi degli extra).

Per semplicità ed efficienza, lo stato dei posti occupati è gestito tramite un array `bookedSeats` direttamente nel documento del volo, contenente il numero del posto e la classe.

Sono stati implementati controlli per:

- Evitare la duplicazione dei posti occupati (validazione sul set dei posti).
- Assicurare la coerenza temporale (l'orario di arrivo deve essere successivo a quello di partenza).
- `flightNumber` : Stringa (Codice volo).

- `company` : ObjectId (Riferimento a `User`).
- `airplane` : ObjectId (Riferimento a `Airplane`).
- `route` : ObjectId (Riferimento a `Route`).
- `departureTime` : Data.
- `arrivalTime` : Data.
- `active` : Booleano.
- `prices` : Oggetto con i prezzi base per classe (`economy` , `business` , `firstclass`) e prezzi extra (`baggage` , `legroom` , `priorityBoarding`).
- `bookedSeats` : Array di oggetti per tracciare i posti occupati. Ogni oggetto contiene:
 - `seat` : Stringa (es. "1A").
 - `travelClass` : Stringa (La classe in cui si trova il posto).

Tickets

```
const mongoose = require('mongoose');
const ticketSchema = new mongoose.Schema({
  user :{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },

  flight :{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Flight',
    required: true
  },

  seat :{
    type: String,
    uppercase: true,
    required: true,
  },

  class : {
    type: String,
    enum: ['economy', 'business', 'firstclass'],
    required: true
  },

  extras: {
    baggage: {
      type: Boolean,
      default: false,
    },
  },
});
```

```

    legroom: {
      type: Boolean,
      default: false
    },
    priorityBoarding: {
      type: Boolean,
      default: false
    }
  },
  price: {
    type: Number,
    required: true,
    min : 0
  }
});

ticketSchema.pre('save', async function(next) {
  // pre significa prima di salvare
  // alla funzione passo next che mi dice che posso andare avanti
  let total=0;
  const Flight = mongoose.model('Flight');
  const flightDoc = await Flight.findById(this.flight);
  if (!flightDoc) {
    throw new Error('Volo non trovato, impossibile calcolare il
prezzo del biglietto');
  }
  total += flightDoc.prices[this.class];
  //mi estrapolo il prezzo base in base alla classe
  total += this.extras.baggage ? flightDoc.prices.extras.baggage : 0;
  //se true aggiungo il prezzo altrimenti 0
  total += this.extras.legroom ? flightDoc.prices.extras.legroom : 0;
  total += this.extras.priorityBoarding ?
flightDoc.prices.extras.priorityBoarding : 0;
  this.price = total;
  next();
});

module.exports = mongoose.model('Ticket', ticketSchema);

```

In questo schema, l'associazione tra utente e volo rappresenta il **biglietto effettivo**. Oltre ai riferimenti agli ID dell'utente e del volo, vengono memorizzati il posto assegnato e il prezzo finale. Quest'ultimo è stato incluso nell'entità per semplicità e per gestire l'aggiunta di eventuali extra. A livello logico, è implementata una funzione automatizzata per il calcolo dei prezzi che esegue un doppio compito: verifica l'esistenza del volo (validazione) e calcola l'importo totale del biglietto includendo le opzioni selezionate.

- `user` : ObjectId (Riferimento a `User` -> Passeggero).
- `flight` : ObjectId (Riferimento a `Flight`).
- `seat` : Stringa (Il posto assegnato, es. "12B").
- `class` : Stringa (Enum: 'economy', 'business', 'firstclass').
- `price` : Numero (Prezzo finale calcolato).
- `extras` : Oggetto booleano per i servizi aggiuntivi (`baggage` , `legroom` , `priorityBoarding`).

Bookings

```
const mongoose = require('mongoose');
const bookingSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },

  tickets: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Ticket',
    required: true
  }],

  totalPrice: {
    type: Number,
    required: true
  },

  status: {
    type: String,
    enum: ['confirmed', 'cancelled', 'used'],
    default: 'confirmed'
  },

  bookingDate: { //quando è stato effettuata la prenotazione
    type: Date,
    default: Date.now
  }

});

module.exports = mongoose.model('Booking', bookingSchema);
```

La collezione `Booking` funge da aggregatore, rappresentando la transazione completa. Un singolo oggetto `Booking` contiene un array di riferimenti ai biglietti (`tickets`), permettendo

di gestire in un'unica entità voli con scalo.

- `user` : ObjectId (Riferimento a `User` che ha effettuato la prenotazione).
- `tickets` : Array di ObjectId (Riferimento ai documenti nella collezione `tickets`).
- `totalPrice` : Numero (Somma dei prezzi dei biglietti).
- `status` : Stringa (Enum: 'confirmed', 'cancelled', 'used').
- `bookingDate` : Data (Data di acquisto).

Rest API

L'interazione tra il client Angular e il server Node.js è mediata da un'architettura API RESTful. Le comunicazioni si basano sul protocollo HTTP e utilizzano il formato JSON per lo scambio dati. Il backend espone diversi endpoint che il client interroga per recuperare le informazioni necessarie al popolamento del frontend. La sicurezza è garantita da un sistema di autenticazione stateless tramite JWT (approfondito successivamente), che consente di mantenere lo stato di autenticazione dell'utente evitando richieste ridondanti per la verifica della sessione. Il token JWT, archiviato localmente, viene allegato all'header delle richieste HTTP per validare l'identità dell'utente e applicare i relativi controlli sui ruoli e permessi. Tutte questi endpoints si appoggiano a dei controller presenti nel Server che gestiscono la logica di inserimento e prelievo dai dati dal DB.

- **Auth Controller:** Gestisce l'intero ciclo di vita dell'utente, inclusa la registrazione, l'autenticazione tramite token JWT, l'aggiornamento dei profili e la cancellazione degli account.
- **Flight Controller:** Fornisce le funzionalità per la ricerca pubblica dei voli (anche con filtri complessi) e permette agli amministratori o alle compagnie aeree di creare, aggiornare e gestire la disponibilità dei voli.
- **Booking Controller:** Gestisce il processo di prenotazione, permettendo ai passeggeri di creare ordini per voli singoli o con scalo, occupando i posti e calcolando il prezzo totale.
- **Ticket Controller:** Si occupa dell'emissione e della visualizzazione dei biglietti singoli. Permette il recupero dei ticket filtrati per utente o volo, garantendo che ogni passeggero veda solo i propri dati.
- **Airport & Airplane Controllers:** Gestiscono le anagrafiche di base. L'Airport Controller permette la ricerca e l'inserimento degli scali, mentre l'Airplane Controller gestisce la flotta di aeromobili e la loro capacità.
- **Routes Controller:** Definisce le tratte aeree (collegamenti tra due aeroporti) che le compagnie aeree possono poi utilizzare per programmare i voli effettivi.
- **Helper Controller:** Una suite di funzioni di supporto condivise che gestiscono operazioni comuni come la risoluzione degli ID aeroportuali dai codici, il rilascio dei posti e la logica di cancellazione sicura.

```
const airplaneRoutes = require('./routes/airplaneRoutes');
const airportRoutes = require('./routes/airportRoutes');
```

```
const authRoutes = require('./routes/authRoutes');
const bookingRoutes = require('./routes/bookingRoutes');
const flightsRoutes = require('./routes/flightsRoutes');
const routeRoutes = require('./routes/routeRoutes');
const ticketsRoutes = require('./routes/ticketRoutes');
```

Auth route

1. Register

Endpoint: POST /api/auth/register

crea un nuovo utente

Requisiti:

- nessuno

```
Richiesta: POST /api/auth/register
{
  email: 'pietro.canale@gmail.com',
  password: '123456',
  role: 'passenger',
  name: 'Pietro',
  surname: 'Canale',
  birthdate: '1990-10-26',
  phonenumber: '123456',
  paymentAddress: "via dell'abete 5 Bassano"
}

//esempio di dati reicevuti per creare Airline
{
  email: 'airline@airline.it',
  password: '123456',
  role: 'airline',
  name: '',
  surname: '',
  birthdate: '',
  phonenumber: '',
  paymentAddress: '',
  company: 'New Ariline'
}

//RISPOSTE

{ //successo
  "error": false,
  "errorMessage": "",
  "message": "Nuovo utente registrato con successo"
}
```

```

{ //email duplicata
  "error": true,
  "errorMessage": "Email già in uso nel db"
}

{ //messaggio nel caso non sia arrivato uno dei due dati
  "error": true,
  "errorMessage": "manca email o password"
}

{ //errore nel try catch
  "error": true,
  "errorMessage": "Server error during registration"
}

```

2. Login

Endpoint: POST /api/auth/login

Autentica l'utente e restituisce un JWT

Requisiti:

- Basic Auth (email:password nell' Authorization header)

```

POST /api/auth/login
Headers: {
  host: 'localhost:3005',
  connection: 'keep-alive',
  'sec-ch-ua-platform': '"Windows"',
  authorization: 'Basic YW5kcmVhLmdpYXp6b25AZ21haWwuY29tOjEyMzQ1Ng==',
  //qua abbiamo email e password in base64
  'cache-control': 'no-cache',
  'sec-ch-ua': '"Not(A:Brand";v="8", "Chromium";v="144", "Google
Chrome";v="144"',
  'sec-ch-ua-mobile': '?0',
  'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36',
  accept: 'application/json, text/plain, */*',
  'content-type': 'application/x-www-form-urlencoded',
  origin: 'http://localhost:4200',
  'sec-fetch-site': 'same-site',
  'sec-fetch-mode': 'cors',
  'sec-fetch-dest': 'empty',
  referer: 'http://localhost:4200/',
  'accept-encoding': 'gzip, deflate, br, zstd',
  'accept-language': 'it-IT,it;q=0.9,en-US;q=0.8,en;q=0.7'
}

//risultato positivo

```

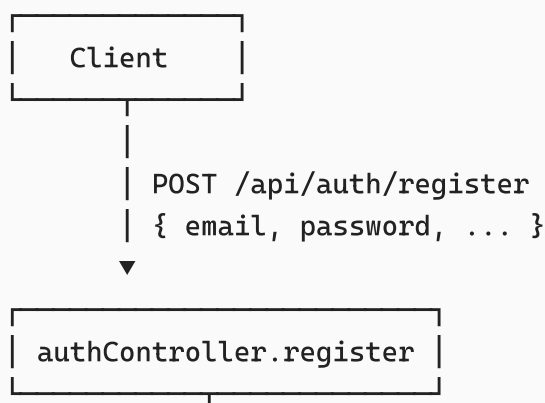
```
{
  "error": false,
  "errorMessage": "",
  "token":
    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6ImFuZHJLYS5naWF6em9uQGdtYWlsLmNvbSI6InJvbmGU0iOiJwYXNzZW5nZXIiLCJpZCI6IjY5OGU4Y2Y3NzdjYzczZjE5ZTAzYzI1NSIsIm5hbWUiOiJBbmRyZWEiLCJzdBXAuYmVtYmVyIjoimzM5MTU5MjkxOSIsInBheW1lbnRBZGRyZXNzIjoimVmlhIENhc29uZXR0byAxNjciciLCJkYXRlb2ZjcmVhdGlvdBI6IjIwMjYyMDItMTNUMDI6MTkuNjU1WiIsImVudCI6MTc3MTIwNjE5MScwiZXBhIjojb2NzcxfQ.jha4-nqB9U9KlcKLrfix6k_1mD-9rLmYdF1Xb7jCkWk"
}

//risultato negativo
{
  "headers": {
    "normalizedNames": {},
    "lazyUpdate": null
  },
  "status": 401,
  "statusText": "Unauthorized",
  "url": "http://localhost:3005/api/auth/login",
  "ok": false,
  "name": "HttpErrorResponse",
  "message": "Http failure response for http://localhost:3005/api/auth/login: 401 Unauthorized",
  "error": "Unauthorized"
}
```

Autenticazione: Passport.js & JWT

L'autenticazione del sistema è **ibrida**: utilizza **Passport.js** con strategia *Basic Auth* per convalidare il login iniziale, dopodiché si basa su **JSON Web Token (JWT)** per gestire l'autorizzazione nelle richieste successive. Le password non sono mai salvate in chiaro nel database, ma vengono sottoposte a un processo di **hashing** e salting.

Workflow registrazione



```
|
|→ Validazione campi obbligatori
|   (email, password)
|
|→ Controllo email duplicata
|   User.findOne({ email })
|
|→ Creazione nuovo User
|   new User(req.body)
|
|→ Hashing password
|   user.setPassword(password)
|   |→ Genera salt random
|   |→ Crea hmac con sha512
|
|→ Salvataggio nel DB
|   user.save()
|
|→ Risposta al client
|   { message: "Utente registrato" }
```

Il processo inizia quando un ipotetico utente decide di registrarsi compilando i dati nel frontend. Queste informazioni vengono inviate alla nostra API tramite una richiesta HTTP POST, strutturata come segue:

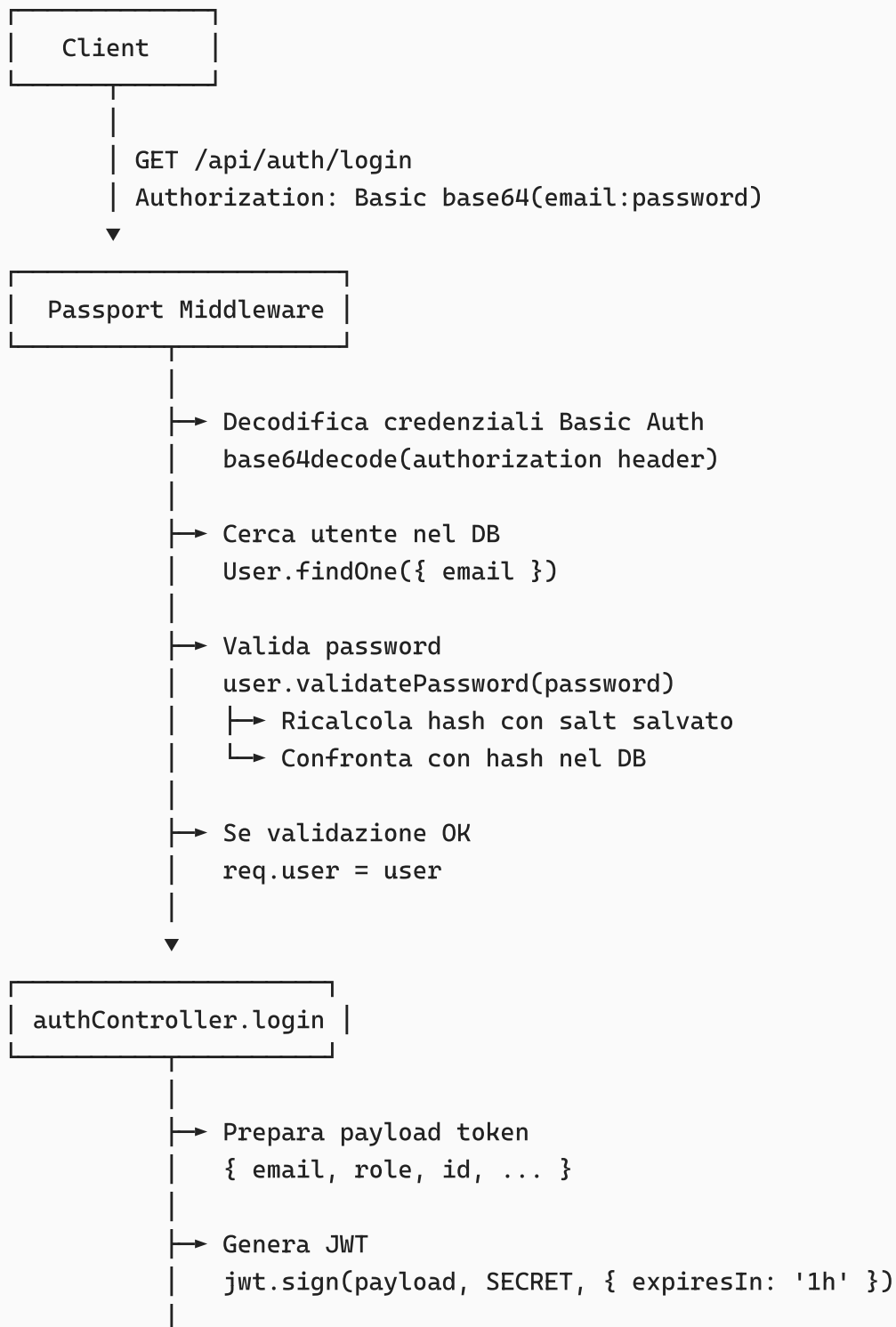
```
// Richiesta HTTP
POST /api/auth/register
Content-Type: application/json
{
  "email": "mario.rossi@example.com",
  "password": "Password123!",
  "role": "passenger",
  "name": "Mario",
  "surname": "Rossi",
  "birthdate": "1990-05-15",
  "phonenumber": "+39 123 456 7890",
  "paymentAddress": "Via Roma 1, Milano"
}
```

L'**authController** verifica immediatamente la presenza dei campi obbligatori (email e password) e controlla nel database che l'email non sia già in uso; in caso contrario, restituisce un errore. Se i controlli passano, viene creato un nuovo oggetto utente, ma la password richiede un trattamento speciale prima del salvataggio. Tramite una funzione del modello User, il sistema genera un **salt** (una stringa casuale di 16 byte) e lo utilizza insieme alla password per creare un hash crittografico tramite algoritmo HMAC. Nel database vengono salvati sia il salt che il risultato dell'hashing (il **digest**), permettendo così di

verificare l'identità in futuro senza mai conoscere la password originale. Solo a questo punto l'utente viene salvato effettivamente e il client riceve conferma.

```
{
  "error": false,
  "errorMessage": "",
  "message": "Nuovo utente registrato con successo"
}
```

Workflow login



```
↳ Risposta al client
{ token: "eyJhbGciOi..." }
```

Per il login, la rotta è protetta dal middleware **Passport** configurato con la strategia Basic.

```
router.get('/login', passport.authenticate('basic', { session: false }),
authController.login);
```

Il client invia le credenziali codificate in Base64 all'interno dell'header `Authorization`, operazione che deve avvenire su HTTPS, altrimenti non sicura.

```
Authorization: Basic bWFyaW9AZW1haWwuaXQ6cGFzc3dvcmQxMjM
```

Passport intercetta la richiesta, decodifica l'header per ottenere email e password in chiaro e cerca l'utente nel database. Se l'utente esiste, il sistema recupera il salt salvato e ricalcola l'hash della password appena fornita: se il risultato coincide con il digest nel database, le credenziali sono valide. A questo punto, Passport passa il controllo al controller di login, il quale prepara un payload con i dati dell'utente.

```
const tokendata = {
  email: user.email,
  role: user.role,
  id: user._id,
  // Dati Passenger
  name: user.name,
  surname: user.surname,
  birthdate: user.birthdate,
  phonenumber: user.phonenumber,
  paymentAddress: user.paymentAddress,

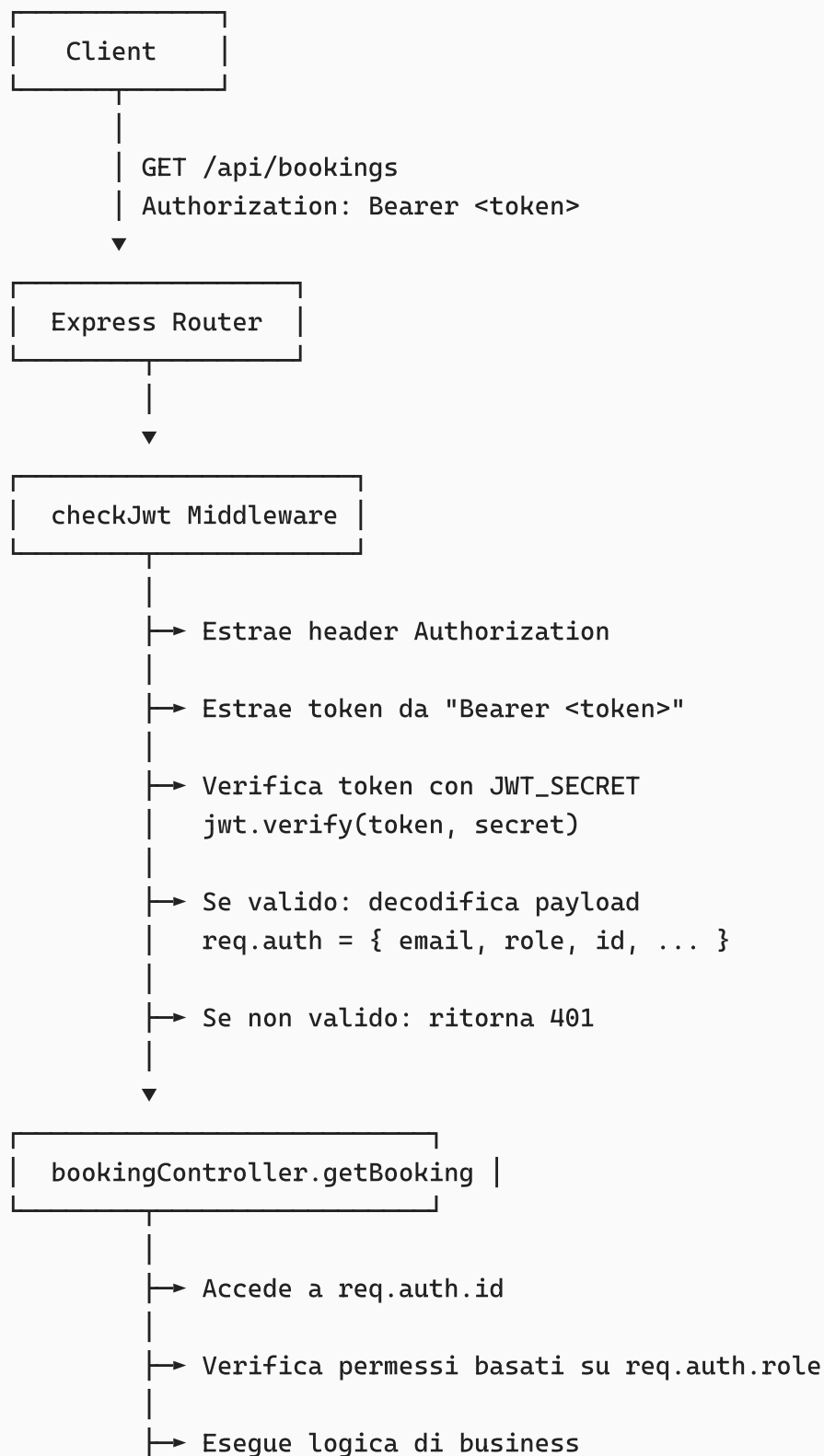
  // Dati Airline
  company: user.company,
  // Info di sistema
  dateofcreation: user.dateofcreation
};
```

```
{
  "error": false,
  "errormessage": "",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Questi dati vengono codificati in Base64 (quindi leggibili, ma non modificabili) e firmati digitalmente utilizzando una chiave segreta (**JWT_SECRET**). Il token risultante, che garantisce l'autenticità dei dati, viene inviato al client che lo salva per le richieste future.

Richieste a rotte protette

Una volta ottenuto il token, il client deve includerlo in ogni richiesta successiva verso rotte protette.



└─ Risposta al client

La richiesta HTTP include l'header `Authorization` con il prefisso "Bearer".

```
POST /api/bookings HTTP/1.1
Host: localhost:3000
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6Im1hcm1vLi4uIn0.Sf1Kxw...
Content-Type: application/json

{
  "flightId": "AZ123",
  "seat": "12A"
}
```

Il nostro middleware personalizzato `authMiddleware.js` intercetta la richiesta prima di passarla al controllore protetto dal requisito di autenticazione. Viene rimosso dal token il prefisso "Bearer" e la stringa restante viene verificata utilizzando la chiave segreta. Se la firma è valida e il token non è scaduto, il payload viene decodificato e inserito nell'oggetto `req.auth`. In questo modo, i controller successivi possono accedere direttamente ai dati dell'utente (come ID o ruolo) per eseguire la logica in sicurezza, senza dover ripetere l'autenticazione sul database

3. Patch user

```
router.patch('/changeData', checkJwt, authController.changeData);
```

Endpoint: `PATCH /api/auth/users`

Descrizione: cambia le informazioni base degli account e cambia anche la password se indicata.

Se la password viene cambiata si genera anche un nuovo JWT

necessario:

- Autorizzazione con JWT e stesso id dell'account da modificare

Request Body:

```
Richiesta: PATCH /api/auth/users
{ //richiesta per user
  id: '698e8cf777cc73f19e03c255',
  name: 'Fabio',
  surname: 'Verdi',
```

```

    phonenumber: '339111111',
    paymentAddress: 'Via dei pioppi 7 Belluno',
    birthdate: '2022-07-08',
    newPassword: '1234567',
    oldPassword: '123456'
  }

  { //per airline
    id: '69862a0d94b164880e956372',
    phonenumber: '12346667',
    paymentAddress: 'Via dei pioppi 9 Firenze',
    company: 'Miao Airlines',
    newPassword: '123456',
    oldPassword: '12345678'
  }

  { //errore
    "error": true,
    "errorMessage": "Vecchia password non corretta"
  }

  {
    "error": true,
    "errorMessage": "Errore durante il cambio dei dati"
  }
}

```

4. Get Users

Endpoint: GET /api/auth/users

Requisiti:

- Autorizzazione con JWT e ruolo di Admin
- parametri della query
 - name (opzionale)
 - surname (opzionale)
 - role (opzionale)
 - email (opzionale)
 - birthdate (opzionale)

dal lato backend è predisposta per filtrare per diversi parametri ma per semplicità il frontend sfrutta solo la selezione per email

```

//richiesta
GET /api/auth/users?email=andrea.giazzon@gmail.com

```

```
//messaggio con filtro per mail
{
  "error": false,
  "errorMessage": "",
  "users": [
    {
      "_id": "698e8cf777cc73f19e03c255",
      "email": "andrea.giazzon@gmail.com",
      "role": "passenger",
      "name": "Fabio",
      "surname": "Verdi",
      "birthdate": "2022-07-20T00:00:00.000Z",
      "phonenumber": "339111111",
      "paymentAddress": "Via dei pioppi 7 Belluno",
      "dateofcreation": "2026-02-13T02:31:19.655Z",
      "__v": 0
    }
  ]
}

//errore se non ho i permessi necessari di admin
{
  "error": true,
  "errorMessage": "Non hai i permessi per visualizzare gli utenti"
}

//errore server nel recuperare i dati
{
  "error": true,
  "errorMessage": "Errore durante il recupero degli utenti"
}
```

5. Delete User

Endpoint: DELETE /api/auth/Users/:id

Gestisce la procedura di cancellazione di un account utente e delle entità ad esso associate.

L'endpoint implementa una logica differenziata in base al ruolo dell'utente per garantire

l'integrità dei dati storici:

- **Passeggeri:** Esegue una cancellazione fisica (Hard Delete) dell'account, delle prenotazioni e dei biglietti associati.
- **Compagnie Aeree:** Esegue una cancellazione dell'account di accesso, ma applica un **Soft Delete** (disattivazione logica) ai Voli e alle Rotte gestite dalla compagnia. Questo preserva lo storico delle operazioni nel database impostando un flag di stato (es. `active: false`) invece di rimuovere i record.

Requisiti:

- Autorizzazione con JWT e ruolo di Admin
- ID utente

//richiesta

DELETE /api/auth/users/699272ba39ad3edf2c7fbcd7

//risposta positiva

```
{
  "error": false,
  "errormessage": "",
  "message": "account cancellato con successo",
  "user": {
    "_id": "699272ba39ad3edf2c7fbcd7",
    "email": "pietro.canale@gmail.com",
    "role": "passenger",
    "name": "Pietro",
    "surname": "Canale",
    "birthdate": "1990-10-26T00:00:00.000Z",
    "phonenumber": "123456",
    "paymentAddress": "via dell'abete 5 Bassano",
    "dateofcreation": "2026-02-16T01:28:26.829Z",
  }
}
```

//possibili errori con risposta negativa

```
{
  "error": true,
  "errormessage": "non hai i permessi per cancellare questo account"
}

{
  "error": true,
  "errormessage": "Utente non trovato"
}

{
  "error": true,
  "errormessage": "non puoi cancellare un account admin"
}

{
  "error": true,
  "errormessage": "Errore durante la cancellazione dell'account"
}
```

Airport Endpoints

6. Get Airports

Endpoint: `GET /api/airports`

Restituisce una lista di aeroporti filtrati in base ai parametri passati nella query string dell'URL. Se non viene passato nessun parametro, restituisce tutti gli aeroporti.

Requisiti:

- Autorizzazione di nessun tipo
- parametri della query:
 - `code` (opzionale) codice aeroporto
 - `name` (opzionale) nome aeroporto
 - `city` (opzionale) città aeroporto
 - `country` (opzionale) nazione aeroporto

```
//Richiesta
GET /api/airports/?city=Italia

//risposta
{
  "message": "Aeroporti trovati!",
  "airports": [
    {
      "active": true,
      "_id": "69862a0e94b164880e95637d",
      "code": "LIN",
      "name": "Milano Linate",
      "city": "Milano",
      "country": "Italia",
      "__v": 0
    },
    {
      "active": true,
      "_id": "69862a0e94b164880e95637e",
      "code": "FCO",
      "name": "Roma Fiumicino",
      "city": "Roma",
      "country": "Italia",
      "__v": 0
    },
    {
      "_id": "698bbf93cc8519fdb699451c",
      "code": "NEM",
```

```
        "name": "nemeggio",
        "city": "Feltre",
        "country": "italia",
        "active": true,
        "__v": 0
      }
    ]
  }

  //errore
  {
    "error": true,
    "errorMessage": "Errore recupero aeroporti"
  }
```

7. Create Airport

Endpoint: POST /api/airports

Consente solo agli admin di creare aeroporti che possono essere poi usati dalle compagnie aeree

Requisiti:

- Autorizzazione con JWT e ruolo di Admin

```
//Richiesta
POST /api/airports/

//body della richiesta JSON
{
  "code": "VEN",
  "name": "Marco Polo",
  "city": "Venezia",
  "country": "Italia"
}

{ //risposta successo
  "message": "Nuovo aeroporto creato!",
  "airport": {
    "code": "VEN",
    "name": "Marco Polo",
    "city": "Venezia",
    "country": "Italia",
    "active": true,
```

```
    "_id": "6993197b49c35bb91534a25d",
    "__v": 0
  }
}

{ //possibili codici di errore
  "error": true,
  "errorMessage": "Non sei un admin non puoi creare un aeroporto"
}

{
  "error": true,
  "errorMessage": "Non sei un admin non puoi creare un aeroporto"
}

{
  "error": true,
  "errorMessage": "Errore interno durante la creazione dell'aeroporto"
}
```

Airplane Endpoints

8. Get Airplanes

Endpoint: GET /api/airplanes

Cerca aerei nel database filtrando per modello. **Nota:** Se il parametro `airplaneModel` non è presente, l'API **non esegue la ricerca** e restituisce immediatamente una lista vuota.

Requisiti:

- **Autorizzazione:** Nessuna (Pubblico)
- **Query Params:**
 - `airplaneModel` : **Necessario per ottenere risultati** (es. "Boeing"). Se omesso, restituisce array vuoto.

GET /api/airplanes/?airplaneModel=Boeing

```
{
  "message": "Modello di aereo trovato!",
  "airplanes": [
    {
      "capacity": {
        "economy": {
          "rows": 20,
          "seatsPerRow": 6,
```

```

        "seatLetters": "ABCDEF",
        "_id": "69862a0e94b164880e956383",
        "numberOfSeats": 120
    },
    "business": {
        "rows": 5,
        "seatsPerRow": 4,
        "seatLetters": "ACDF",
        "_id": "69862a0e94b164880e956384",
        "numberOfSeats": 20
    },
    "firstclass": {
        "rows": 0,
        "seatsPerRow": 0,
        "seatLetters": "",
        "_id": "69862a0e94b164880e956385",
        "numberOfSeats": 0
    }
},
_id": "69862a0e94b164880e956382",
airplaneModel": "Boeing 737-800",
totalSeats": 140,
active": true,
__v": 0
}
]
}

// Status: 200 OK
{
    "message": "Modello di aereo trovato!",
    "airplanes": []
}

{
    "message": "nessun filtro applicato!",
    "airplanes": []
}

{//errore
    "error": true,
    "errorMessage": "Errore recupero aerei"
}

```

9. Create Airplane

Endpoint: POST /api/airplanes

Consente di creare un nuovo modello di aereo configurando le file e i posti per ogni classe. Il sistema calcola automaticamente le lettere dei sedili e i totali.

necessario:

- Autorizzazione con JWT
- Ruolo: **Admin** oppure **Airline**

```
// Richiesta
POST /api/airplanes/

// Body della richiesta JSON
{
  "airplaneModel": "Aereo di Prova",
  "capacity": {
    "economy": {
      "rows": 4,
      "seatsPerRow": 10
    },
    "business": {
      "rows": 5,
      "seatsPerRow": 4
    },
    "firstclass": {
      "rows": 6,
      "seatsPerRow": 3
    }
  }
}

// Risposta successo
{
  "message": "Nuovo aereo creato!",
  "airplane": {
    "airplaneModel": "Aereo di Prova",
    "capacity": {
      "economy": {
        "rows": 4,
        "seatsPerRow": 10,
        "seatLetters": "ABCDEFGH IJ",
        "numberOfSeats": 40,
        "_id": "6993260784143febca81969d"
      },
      "business": {
        "rows": 5,
        "seatsPerRow": 4,
        "seatLetters": "ABCD",
        "numberOfSeats": 20,
        "_id": "6993260784143febca81969e"
      }
    }
  }
}
```

```

        },
        "firstclass": {
            "rows": 6,
            "seatsPerRow": 3,
            "seatLetters": "ABC",
            "numberOfSeats": 18,
            "_id": "6993260784143febca81969f"
        }
    },
    "totalSeats": 78,
    "active": true,
    "_id": "6993260784143febca81969c",
    "__v": 0
}

// Errori possibili
{
    "error": true,
    "errorMessage": "Non autorizzato a creare un aereo"
}

{
    "error": true,
    "errorMessage": "Codice aereo già esistente"
}

{
    "error": true,
    "errorMessage": "Errore nel salvataggio"
}

```

10. Delete Airplane

Endpoint: DELETE /api/airplanes/:id

Effettua una cancellazione logica (Soft Delete) impostando `active: false`. Inoltre, cancella a cascata tutti i voli associati a questo aereo e non comparirà più nelle ricerche.

necessario:

- Autorizzazione con JWT e ruolo di **Admin** (Le compagnie aeree NON possono eliminare aerei, solo crearli).
- ID dell'aereo nell'URL.

```
//richiesta
```

```
DELETE /api/airplanes/6993259649c35bb91534a266
```

```
//risposta positiva
```

```
{
  "message": "Aereo eliminato con successo",
  "airplane": {
    "capacity": {
      "economy": {
        "rows": 5,
        "seatsPerRow": 4,
        "seatLetters": "ABCD",
        "numberOfSeats": 20,
        "_id": "6993259649c35bb91534a267"
      },
      "business": {
        "rows": 5,
        "seatsPerRow": 5,
        "seatLetters": "ABCDE",
        "numberOfSeats": 25,
        "_id": "6993259649c35bb91534a268"
      },
      "firstclass": {
        "rows": 7,
        "seatsPerRow": 6,
        "seatLetters": "ABCDEF",
        "numberOfSeats": 42,
        "_id": "6993259649c35bb91534a269"
      }
    },
    "_id": "6993259649c35bb91534a266",
    "airplaneModel": "Aereo Prova",
    "totalSeats": 87,
    "active": false,
    "__v": 0
  }
}
```

```
//errori possibili
```

```
{
  "error": true,
  "errorMessage": "Non sei un admin non puoi eliminare un aereo"
}
```

```
{
  "error": true,
  "errorMessage": "Aereo non trovato"
}
```

```
{
  "error": true,
  "errorMessage": "Errore durante l'eliminazione dell'aereo",
  "details": "Messaggio di errore tecnico qui"
}
```

Route Endpoints

11. Get Routes

Endpoint: GET /api/routes

Cerca una rotta specifica registrata per la compagnia aerea loggata, basandosi sugli aeroporti di partenza e arrivo.

Requisiti

- Autorizzazione con JWT
- Ruolo: **Airline** oppure **Admin**
- Parametri della query:
 - `from` (obbligatorio): Codice aeroporto di partenza (es. FCO)
 - `to` (obbligatorio): Codice aeroporto di arrivo (es. JFK)

```
//Richiesta
GET /api/routes/?from=LIN&to=JFK

{ //richiesta andata a buon fine
  "_id": "6993391a84143febca8196be",
  "airlineId": "69862a0d94b164880e956373",
  "departureAirport": {
    "active": true,
    "_id": "69862a0e94b164880e95637d",
    "code": "LIN",
    "name": "Milano Linate",
    "city": "Milano",
    "country": "Italia",
    "__v": 0
  },
  "arrivalAirport": {
    "active": true,
    "_id": "69862a0e94b164880e95637f",
    "code": "JFK",
    "name": "John F. Kennedy",
    "city": "New York",
    "country": "USA",
  }
}
```

```
        "__v": 0
      },
      "active": true,
      "__v": 0
    }

    // Errori possibili
    {
      "error": true,
      "errorMessage": "Solo le compagnie aeree possono cercare le proprie rotte"
    }

    {
      "error": true,
      "errorMessage": "La rotta FCO-JFK non è registrata per la tua compagnia."
    }

    {
      "error": true,
      "errorMessage": "Errore ricerca rotte",
      "details": "..."
    }
  }
}
```

12. Create Route

Endpoint: POST /api/routes

Consente di creare una nuova tratta aerea collegando due aeroporti esistenti.

Requisiti:

- Autorizzazione con JWT
- Ruolo: **Airline** oppure **Admin**

Richiesta: POST /api/routes/

```
// Body della richiesta JSON
{
  "departureCode": "LIN",
  "arrivalCode": "JFK"
}

{//richiesta andata a buon fine
  "message": "Nuova tratta aerea creata!",
  "route": {
```

```

        "airlineId": "69862a0d94b164880e956373",
        "departureAirport": "69862a0e94b164880e95637d",
        "arrivalAirport": "69862a0e94b164880e95637e",
        "active": true,
        "_id": "69933eea84143febca8196cf",
        "__v": 0
    }
}

//errori
{
    "error": true,
    "errorMessage": "Solo le compagnie aeree possono creare rotte"
}

{
    "error": true,
    "errorMessage": "Codici aeroporti mancanti"
}

{
    "error": true,
    "errorMessage": "Uno degli aeroporti non esiste"
}

{
    "error": true,
    "errorMessage": "Questa tratta esiste già per la tua compagnia"
}

```

13. Delete Route

Endpoint: DELETE /api/routes/:id

Effettua una cancellazione logica (Soft Delete) impostando `active: false`. Inoltre, lancia un processo per cancellare a cascata tutti i biglietti e i voli futuri associati a questa rotta.

Requisiti:

- Autorizzazione con JWT
- Ruolo: **Admin** oppure il **Proprietario della rotta (Airline)**
- ID della rotta nell'URL

```

// Richiesta
DELETE /api/routes/6993260784143febca819700

// Risposta successo

```

```
{
  "message": "Rotta cancellata",
  "route": {
    "_id": "699348c2b92d8935576f9e39",
    "airlineId": "6992736739ad3edf2c7fbcdcb",
    "departureAirport": "69862a0e94b164880e95637d",
    "arrivalAirport": "69862a0e94b164880e95637f",
    "active": true,
    "__v": 0
  }
}

// Errori possibili
{
  "error": true,
  "message": "Rotta non trovata"
}

{
  "error": true,
  "message": "Non puoi cancellare rotte di altri"
}

{
  "error": true,
  "message": "Errore server durante la cancellazione"
}
```

Flight Endpoints

14. Get Flights

Restituisce una lista di voli filtrati in base ai parametri passati. La ricerca può restituire voli diretti o con scalo.

Requisiti:

- **Autorizzazione:** Nessuna (Pubblico)
- **Parametri della query:**
 - `flightNumber` (opzionale): Numero del volo
 - `from` (opzionale): Aeroporto di partenza (codice/nome/città)
 - `to` (opzionale): Aeroporto di arrivo (codice/nome/città)
 - `initialDate` (opzionale): Data minima partenza (ISO 8601)
 - `finalDate` (opzionale): Data massima partenza (ISO 8601)
 - `company` (opzionale): Nome della compagnia aerea

```
//richiesta
```

```
GET /api/flights/?active=true&from=milano&to=Venezia&initialDate=2026-02-17T00:00:00.000Z
```

```
{
  "message": "risultato ricerca voli!",
  "result": [
    {
      "type": "direct",
      "flights": [
        {
          "prices": {
            "extras": {
              "baggage": 50,
              "legroom": 40,
              "priorityBoarding": 200
            },
            "economy": 10,
            "business": 20,
            "firstclass": 420
          },
          "_id": "69934d2ab92d8935576f9e87",
          "flightNumber": "AZ123",
          "company": {
            "_id": "6992736739ad3edf2c7fbcdB",
            "email": "airline@airline.it",
            "role": "airline",
            "company": "New Ariline"
          },
          "departureTime": "2026-02-17T21:02:00.000Z",
          "arrivalTime": "2026-02-18T05:04:00.000Z",
          "airplane": {
            "capacity": {
              "economy": {
                "rows": 5,
                "seatsPerRow": 4,
                "seatLetters": "ABCD",
                "numberOfSeats": 20,
                "_id": "6993259649c35bb91534a267"
              },
              "business": {
                "rows": 5,
                "seatsPerRow": 5,
                "seatLetters": "ABCDE",
                "numberOfSeats": 25,
                "_id": "6993259649c35bb91534a268"
              },
              "firstclass": {
                "rows": 7,
```

```

        "seatsPerRow": 6,
        "seatLetters": "ABCDEF",
        "numberOfSeats": 42,
        "_id": "6993259649c35bb91534a269"
    },
    {
        "_id": "6993259649c35bb91534a266",
        "airplaneModel": "Aereo Prova",
        "totalSeats": 87,
        "active": false,
        "__v": 0
    },
    "route": {
        "_id": "69934d05b92d8935576f9e7b",
        "airlineId": "6992736739ad3edf2c7fbcd",
        "departureAirport": {
            "active": true,
            "_id": "69862a0e94b164880e95637d",
            "code": "LIN",
            "name": "Milano Linate",
            "city": "Milano",
            "country": "Italia",
            "__v": 0
        },
        "arrivalAirport": {
            "_id": "6993197b49c35bb91534a25d",
            "code": "VEN",
            "name": "Marco Polo",
            "city": "Venezia",
            "country": "Italia",
            "active": true,
            "__v": 0
        },
        "active": true,
        "__v": 0
    },
    "active": false,
    "bookedSeats": [],
    "__v": 0
}
]
}
]
}

```

```
// Errori possibili
```

```

{
    "error": true,
    "errorMessage": "Errore recupero voli"
}

```

```
}
```

15. Create Flight

Endpoint: POST /api/flights

Consente di creare un nuovo volo. Il sistema verifica automaticamente che la rotta esista, che l'aereo sia disponibile e che non ci siano sovrapposizioni orarie.

Requisiti:

- Autorizzazione con JWT
- Ruolo: **Admin** oppure **Airline**

```
//Richiesta:
POST /api/flights/

{ //contenuto del BODY
  "flightNumber": "AZ123",
  "departureTime": "2026-02-17T21:02:00.000Z",
  "arrivalTime": "2026-02-17T22:04:00.000Z",
  "airplaneId": "Aereo Prova",
  "from": "milano",
  "to": "Venezia",
  "prices": {
    "economy": 10,
    "business": 20,
    "firstclass": 30,
    "extras": {
      "legroom": 40,
      "baggage": 50,
      "priorityBoarding": 10
    }
  }
}

{
  "message": "Nuovo volo creato!",
  "flight": {
    "flightNumber": "AZ123",
    "company": "6992736739ad3edf2c7fbcdcb",
    "prices": {
      "economy": 10,
      "business": 20,
      "firstclass": 30,
```

```

        "extras": {
            "baggage": 50,
            "legroom": 40,
            "priorityBoarding": 10
        },
        "departureTime": "2026-02-17T21:02:00.000Z",
        "arrivalTime": "2026-02-17T22:04:00.000Z",
        "airplane": "6993259649c35bb91534a266",
        "route": "69934d05b92d8935576f9e7b",
        "active": true,
        "_id": "69934d2ab92d8935576f9e87",
        "bookedSeats": [],
        "__v": 0
    }
}

// Errori possibili
{
    "error": true,
    "errorMessage": "permessi non sufficienti, non puoi creare un volo"
}

{
    "error": true,
    "errorMessage": "Aereo non trovato o non attivo"
}

{
    "error": true,
    "errorMessage": "L'aereo è già impegnato in un altro volo in questo orario"
}

{
    "error": true,
    "errorMessage": "La rotta specificata non esiste"
}

```

16. Update Flight

Endpoint: PATCH /api/flights/:id

Consente di aggiornare i prezzi, l'orario di partenza e l'orario di arrivo di un volo esistente.

Requisiti:

- Autorizzazione con JWT
- Ruolo: **Admin** oppure il **Proprietario del volo (Airline)**
- ID del volo nell'URL

```
//Richiesta:
PATCH /api/flights/69934d2ab92d8935576f9e87

{ //richiesta nel body
  prices: {
    economy: 10,
    business: 20,
    firstclass: 400,
    extras: { legroom: 40, baggage: 50, priorityBoarding: 200 }
  },
  departureTime: '2026-02-17T21:02:00.000Z',
  arrivalTime: '2026-02-18T05:04:00.000Z'
}

{
  //risultato aggiornamento!
  "message": "Prezzi aggiornati con successo!",
  "flight": {
    "prices": {
      "economy": 10,
      "business": 20,
      "firstclass": 420,
      "extras": {
        "baggage": 50,
        "legroom": 40,
        "priorityBoarding": 200
      }
    },
    "_id": "69934d2ab92d8935576f9e87",
    "flightNumber": "AZ123",
    "company": "6992736739ad3edf2c7fbcdcb",
    "departureTime": "2026-02-17T21:02:00.000Z",
    "arrivalTime": "2026-02-18T05:04:00.000Z",
    "airplane": "6993259649c35bb91534a266",
    "route": "69934d05b92d8935576f9e7b",
    "active": true,
    "bookedSeats": [],
    "__v": 0
  }
}
```

```
// Errori possibili
{
  "error": true,
  "errorMessage": "Volo non trovato"
}
{
  error": true,
  "errorMessage": "Non autorizzato a modificare questo volo"
}
```

17. Delete Flight

Endpoint: DELETE /api/flights/:id

Effettua una cancellazione logica (Soft Delete) impostando `active: false`. Inoltre, cancella tutte le prenotazioni associate a questo volo (Soft Delete)

Requisiti:

- Autorizzazione con JWT
- Ruolo: **Admin** oppure il **Proprietario del volo (Airline)**
- ID del volo nell'URL``

```
//richiesta
DELETE/api/flights/69934d2ab92d8935576f9e87

{ //risposta positiva
  "message": "Volo cancellato",
  "flight": {
    "prices": {
      "extras": {
        "baggage": 50,
        "legroom": 40,
        "priorityBoarding": 200
      },
      "economy": 10,
      "business": 20,
      "firstclass": 420
    },
    "_id": "69934d2ab92d8935576f9e87",
    "flightNumber": "AZ123",
    "company": "6992736739ad3edf2c7fbcdcb",
    "departureTime": "2026-02-17T21:02:00.000Z",
```

```
        "arrivalTime": "2026-02-18T05:04:00.000Z",
        "airplane": "6993259649c35bb91534a266",
        "route": "69934d05b92d8935576f9e7b",
        "active": true,
        "bookedSeats": [],
        "__v": 0
    }
}

//possibili errori
{
    "error": true,
    "errorMessage": "Volo non trovato"
}

{
    "error": true,
    "errorMessage": "Non autorizzato a eliminare questo volo"
}

{
    "error": true,
    "errorMessage": "Errore durante l'eliminazione del volo"
}
```

18. Get Airline Statistics

Endpoint: GET /api/flights/statistics

Restituisce statistiche dettagliate sulle performance della compagnia aerea, calcolando passeggeri totali, ricavi totali, performance per singola rotta (ordinate per numero di passeggeri) e dettagli per singolo aereo/volo.

Requisiti:

- Autorizzazione con JWT
- Ruolo: **Airline** oppure **Admin**
- Parametri della query:
 - `dateFrom` (opzionale): Data di inizio per il calcolo delle statistiche (formato ISO 8601)
 - `dateTo` (opzionale): Data di fine per il calcolo delle statistiche (formato ISO 8601)
 - `email` (opzionale): **Solo per Admin**, permette di filtrare le statistiche per una specifica compagnia aerea tramite la sua email.

```
//chiamata
```

```
GET /api/flights/statistics?dateFrom=2024-01-01&dateTo=2024-12-31
```

```
{{//esempio risposta positiva (admin)
```

```
  "error": false,
  "totalPassengers": 6,
  "totalRevenue": 2935,
  "flightStats": [
    {
      "flightNumber": "LX100",
      "totalPassengers": 2,
      "totalRevenue": 655,
      "numberOfFlights": 1,
      "averageRevenuePerPassenger": 327.5,
      "averagePassengersPerFlight": 2
    },
    {
      "flightNumber": "LX200",
      "totalPassengers": 1,
      "totalRevenue": 1555,
      "numberOfFlights": 1,
      "averageRevenuePerPassenger": 1555,
      "averagePassengersPerFlight": 1
    },
    {
      "flightNumber": "SW500",
      "totalPassengers": 2,
      "totalRevenue": 345,
      "numberOfFlights": 1,
      "averageRevenuePerPassenger": 172.5,
      "averagePassengersPerFlight": 2
    },
    {
      "flightNumber": "SW600",
      "totalPassengers": 1,
      "totalRevenue": 380,
      "numberOfFlights": 1,
      "averageRevenuePerPassenger": 380,
      "averagePassengersPerFlight": 1
    }
  ],
  "routeStats": [
    {
      "routeId": "69947bc531186f12bbdf885e",
      "departureCode": "LIN",
      "arrivalCode": "FCO",
      "departureCity": "Milano",
      "arrivalCity": "Roma",
```

```

        "totalPassengers": 2,
        "totalRevenue": 655
    },
    {
        "routeId": "69947bc531186f12bbdf8860",
        "departureCode": "LIN",
        "arrivalCode": "MUC",
        "departureCity": "Milano",
        "arrivalCity": "Monaco",
        "totalPassengers": 2,
        "totalRevenue": 345
    },
    {
        "routeId": "69947bc531186f12bbdf885f",
        "departureCode": "FCO",
        "arrivalCode": "JFK",
        "departureCity": "Roma",
        "arrivalCity": "New York",
        "totalPassengers": 1,
        "totalRevenue": 1555
    },
    {
        "routeId": "69947bc531186f12bbdf8861",
        "departureCode": "MUC",
        "arrivalCode": "JFK",
        "departureCity": "Monaco",
        "arrivalCity": "New York",
        "totalPassengers": 1,
        "totalRevenue": 380
    }
]
}

// Errori possibili
{
    "error": true,
    "errorMessage": "Errore durante il recupero delle statistiche",
    "details": "..."
}

{
    "error": true,
    "errorMessage": "Permessi insufficienti"
}

```

19. Ticket Endpoints

Nel backend sono presenti, ma attualmente commentati, gli endpoint dedicati alla gestione atomica dei singoli **Ticket** (`/api/tickets`). Questa architettura è stata definita in una fase preliminare di progettazione (API-First), precedente allo sviluppo del frontend. Durante l'implementazione dell'interfaccia utente in Angular, si è optato per un approccio basato sull'entità aggregata **Booking** (Prenotazione), che gestisce più biglietti simultaneamente. Tuttavia, ho scelto di mantenere la struttura dei Ticket nel codice sorgente poiché è predisposta per sviluppi futuri. È importante notare che questi endpoint non sono codice morto, ma condividono la stessa logica del flusso di prenotazione: si appoggiano infatti alle medesime funzioni ausiliarie (`helperController`) attualmente utilizzate dal `bookingController` per la generazione e la gestione dei titoli di viaggio."

Endpoint: `POST /api/tickets`

Endpoint: `GET /api/tickets`

Endpoint: `DELETE /api/tickets/:ticketID`

20. Get Bookings

Endpoint: `GET /api/bookings`

Recupera lo storico delle prenotazioni (che possono contenere più biglietti).

Comportamento per ruolo:

- **Passenger:** Vede solo le proprie prenotazioni.
- **Admin:** Può vedere tutte le prenotazioni o filtrare.
- **Airline:** Accesso negato (403). Le compagnie aeree non possono accedere ai dati aggregati delle prenotazioni utenti.

Requisiti:

- Autorizzazione con JWT
- Parametri della query:
 - `userId` (opzionale, solo Admin)
 - `bookingId` (opzionale)
 - `dateFrom` (opzionale):
 - `dateTo` (opzionale)

```
//Richiesta:
GET /api/bookings/?userId=69947bc531186f12bbdf8846

[//risposta positiva
{
  "_id": "69947bc531186f12bbdf886c",
  "user": {
    "_id": "69947bc531186f12bbdf8846",
```

```
    "email": "mario@test.com",
    "name": "Mario",
    "surname": "Rossi"
  },
  "tickets": [
    {
      "extras": {
        "baggage": false,
        "legroom": false,
        "priorityBoarding": false
      },
      "_id": "69947bc531186f12bbdf8868",
      "user": "69947bc531186f12bbdf8846",
      "flight": {
        "_id": "69947bc531186f12bbdf8863",
        "flightNumber": "LX100",
        "departureTime": "2026-02-18T08:00:00.000Z",
        "arrivalTime": "2026-02-18T09:00:00.000Z",
        "airplane": {
          "capacity": {
            "economy": {
              "rows": 20,
              "seatsPerRow": 6,
              "seatLetters": "ABCDEF",
              "_id": "69947bc531186f12bbdf8855",
              "numberOfSeats": 120
            },
            "business": {
              "rows": 5,
              "seatsPerRow": 4,
              "seatLetters": "ACDF",
              "_id": "69947bc531186f12bbdf8856",
              "numberOfSeats": 20
            },
            "firstclass": {
              "rows": 2,
              "seatsPerRow": 2,
              "seatLetters": "AC",
              "_id": "69947bc531186f12bbdf8857",
              "numberOfSeats": 4
            }
          }
        },
        "_id": "69947bc531186f12bbdf8854",
        "airplaneModel": "Airbus A350 Lux",
        "totalSeats": 144,
        "active": true,
        "__v": 0
      },
      "route": {
        "_id": "69947bc531186f12bbdf885e",
```

```
        "airlineId": "69947bc531186f12bbdf884b",
        "departureAirport": {
            "_id": "69947bc531186f12bbdf884f",
            "code": "LIN",
            "name": "Milano Linate",
            "city": "Milano",
            "country": "Italia",
            "active": true,
            "__v": 0
        },
        "arrivalAirport": {
            "_id": "69947bc531186f12bbdf8850",
            "code": "FCO",
            "name": "Roma Fiumicino",
            "city": "Roma",
            "country": "Italia",
            "active": true,
            "__v": 0
        },
        "active": true,
        "__v": 0
    },
    {
        "seat": "10A",
        "class": "economy",
        "price": 100,
        "__v": 0
    }
],
"totalPrice": 100,
"status": "confirmed",
"bookingDate": "2026-02-17T14:31:33.483Z",
"__v": 0
}

[
{
    "error": true,
    "errorMessage": "una compagnia non può vedere i biglietti"
}

{
    "error": true,
    "errorMessage": "Errore nel recupero biglietti booking"
}
```

21. Create Booking

Endpoint: POST/api/bookings

Crea una nuova prenotazione completa. Supporta biglietto singolo (solo andata) o doppio (andata e ritorno).

Validazioni Logiche Implementate:

1. **Coerenza Aeroporti:** Se è A/R, l'aeroporto di arrivo dell'andata deve coincidere con la partenza del ritorno.
2. **Intervallo Temporale:** Deve esserci un intervallo di almeno **2 ore** tra l'arrivo del primo volo e la partenza del secondo.

Requisiti:

- Autorizzazione con JWT
- Ruolo: **Passenger**

```
//Richiesta
POST /api/bookings/

{ //body della richiesta
  tick1: {
    userId: '69948413a6654ae07d9837cd',
    flightId: '69948413a6654ae07d9837ea',
    seat: '2A',
    flightClass: 'firstclass',
    extras: { baggage: true, legroom: false, priorityBoarding: false },
    price: 530
  },
  tick2: null
}

{ //risposta positiva
  "error": false,
  "errorMessage": "",
  "message": "Prenotazione creata con successo",
  "bookingId": "6994830931186f12bbdf8913"
}

//errori
{
  "error": true,
  "errorMessage": "Solo i passeggeri possono creare prenotazioni"
}
```

```
{
  "error": true,
  "errorMessage": "Non ci sono biglietti"
}

{
  "error": true,
  "errorMessage": "Errore creazione biglietto"
}
```

22. Cancel Booking

Endpoint: DELETE /api/bookings/:bookingId

Permette di cancellare una prenotazione. L'operazione imposta lo stato della prenotazione su `cancelled` e libera i posti a sedere occupati sui relativi voli (attraverso la funzione `seatReleaser`).

Comportamento per ruolo:

- **Passenger:** Può cancellare solo le **proprie** prenotazioni.
- **Admin:** Può cancellare qualsiasi prenotazione.
- **Airline:** Non autorizzato.

Requisiti:

- Autorizzazione con JWT
- ID della prenotazione nell'URL

```
// Richiesta
DELETE /api/bookings/6994830931186f12bbdf8913

{ // Risposta positiva
  "error": false,
  "errorMessage": "",
  "message": "Prenotazione cancellata con successo"
}

// Possibili Errori
{
  "error": true,
  "errorMessage": "bookingId mancante"
}

{
  "error": true,
  "errorMessage": "booking non tuo o permessi insufficienti per essere"
```

```
cancellato"
}

{
  "error": true,
  "errorMessage": "Errore nella cancellazione della prenotazione"
}
```

WebSocket per Aggiornamenti Real-Time

Il sistema implementa WebSocket tramite **Socket.IO** per gestire la sincronizzazione in tempo reale dello stato dei posti occupati nei voli. Questa scelta architetturale è fondamentale per evitare che più utenti tentino di prenotare simultaneamente lo stesso posto.

Architettura a Stanze (Rooms)

Ogni volo è rappresentato come una **stanza indipendente**. Quando un utente visualizza la pagina di prenotazione di un volo specifico, entra automaticamente nella stanza corrispondente (`flight:${flightId}`). Tutti i passeggeri che stanno prenotando lo stesso volo condividono la medesima stanza, permettendo una comunicazione broadcast efficiente e mirata.

Gestione Server-Side

Il server rimane in ascolto di due eventi principali emessi dai client:

- `join-flight` : quando un utente apre la pagina di prenotazione
- `leave-flight` : quando l'utente abbandona la pagina o finalizza la prenotazione

All'ingresso nella stanza, il server conferma l'avvenuta connessione emettendo l'evento `joined-flight` al client.

Ogni volta che viene effettuata un'operazione sui posti (prenotazione o cancellazione), le funzioni helper nel backend emettono automaticamente un evento `seat-update` a tutti i client presenti nella stanza del volo.

Gestione Client-Side

Nel frontend Angular, il servizio `websocket.service.ts` si occupa dell'intera comunicazione WebSocket con il server. Quando l'utente apre la pagina di creazione booking (`booking-creation.component.ts`), il componente:

1. **Si connette al WebSocket** ed entra nella stanza del volo tramite l'evento `join-flight`
2. **Rimane in ascolto** sulla connessione full-duplex per ricevere aggiornamenti in tempo reale tramite sottoscrizione all'observable
3. **Aggiorna la UI dinamicamente** quando riceve eventi

4. **Emette la propria prenotazione** quando l'utente finalizza il booking: la richiesta HTTP POST `/api/bookings` attiva automaticamente lato server l'emissione di `seat-update`, notificando tutti gli altri presenti nella stanza che il posto è ora occupato
5. **Esce dalla stanza** tramite `leave-flight` quando abbandona la pagina

Angular frontend

Pagine/Routes

1. Accesso e Profilo Utente

- **Login (/login)**: Pagina dedicata all'autenticazione degli utenti (Passeggeri, Compagnie Aeree, Admin).
- **Registration (/registration)**: Modulo per la creazione di nuovi account utente.
- **Profile (/profile)**: Area personale dove l'utente loggato può visualizzare i propri dati anagrafici e modificare le informazioni sensibili, inclusa la password.

2. Navigazione e Ricerca (Pubblico / Passeggeri)

- **Home (/home)**: Dashboard principale dinamica che adatta il contenuto in base al ruolo dell'utente:
 - *Ospite/Passeggero*: Interfaccia di ricerca voli (form di inserimento date, partenza, destinazione).
 - *Compagnia Aerea*: Hub gestionale per accedere a voli, rotte, flotta e statistiche.
 - *Admin*: Pannello di controllo per la gestione globale di utenti e infrastrutture (aeroporti).
- **Flight Results (/results)**: Pagina dei risultati di ricerca che visualizza la lista dei voli disponibili corrispondenti ai filtri inseriti dall'utente nella Home.

3. Flusso di Prenotazione (Passeggeri)

- **Booking Creation (/bookingCreation)**: Wizard di prenotazione dove l'utente finalizza l'acquisto del volo selezionato, scegliendo il posto sulla mappa dell'aereo, la classe (Economy, Business, First) ed eventuali servizi aggiuntivi (bagagli, priority).
- **Booking List ("Prenotazioni") (/bookingPrenotation)**: Storico completo delle prenotazioni effettuate dall'utente, con riepilogo dello stato (confermato, cancellato).
- **Booking Detail (/bookingDetail)**: Scheda di dettaglio di una singola prenotazione. Mostra tutte le specifiche del viaggio: codice biglietto, orari, aeroporti, importo pagato e permette (se le condizioni lo consentono) la cancellazione.

4. Area Gestionale (Compagnie Aeree)

- **Airline Routes (/airline-Routes)**: Interfaccia per la definizione delle tratte aeree. Permette di collegare due aeroporti esistenti creando una rotta percorribile.

- **Airline Airplanes** (`/airline-Airplanes`): Gestione della flotta. Consente di registrare nuovi aeromobili e configurare il layout dei sedili (righe e colonne per classe). (disponibile anche per l'admin)
- **Airline New Flight** (`/airline-Flight`): Modulo per la schedulazione di nuovi voli. L'operatore associa una rotta esistente, un aereo e un orario per inserire un volo nel calendario.
- **Airline Manage Flight** (`/airline-Manage`): Pannello per la manutenzione dei voli esistenti. Permette di modificare i dettagli operativi o cancellare voli già programmati.
- **Airline Statistics** (`/airline-stat`): Dashboard analitica che mostra le performance della compagnia (numero passeggeri, ricavi totali, tasso di riempimento voli). (disponibile anche per l'admin)

5. Area Amministrazione (Admin)

- **Admin Create Airports** (`/admin-create-airports`): Pagina per l'inserimento e la gestione dei nodi aeroportuali (Codice IATA, Nome, Città, Nazione) nel database.
- **Admin Add User** (`/admin-add-user`): Interfaccia privilegiata per la creazione manuale di utenti speciali (altri Admin o account Compagnia Aerea) e passeggeri.

Nel codice (`app.routes.ts`) è presente anche un sistema di sicurezza (**Guards**) che protegge queste pagine:

- `PathGuardGuardUser` : Protegge le pagine di prenotazione (solo passeggeri loggati).
- `PathGuardGuardAirline` : Protegge le pagine di gestione voli (solo compagnie aeree).
- `PathGuardGuardAdmin` : Protegge la creazione di aeroporti e utenti (solo admin).

Servizi

- **HttpService** Gestisce il flusso di autenticazione e l'anagrafica utenti. Si occupa di login, registrazione, persistenza/decodifica del token JWT e gestione dei ruoli (Admin, Passeggero, Compagnia). Include metodi amministrativi per la modifica e cancellazione degli utenti. **Endpoint:** `http://localhost:3005/api/auth`
- **WebsocketService** Gestisce la comunicazione real-time tramite Socket.io. Implementa la logica per l'ingresso e l'uscita dalle "stanze" dei voli (`join-flight`) per sincronizzare lo stato dei posti a sedere tra i client connessi. **Endpoint:** `http://localhost:3005` (Root)
- **SearchService** Gestisce le operazioni sul catalogo voli. Lato utente permette la ricerca filtrata (data, tratta, ecc.); lato compagnia gestisce (creazione, modifica prezzi/orari, cancellazione) dei voli. **Endpoint:** `http://localhost:3005/api/flights`
- **BookingService** Gestisce il ciclo di vita delle prenotazioni. Si occupa dell'emissione dei biglietti, del recupero dello storico prenotazioni e delle cancellazioni. Fornisce metodi di utility per generare e renderizzare la configurazione dei posti (righe e colonne) in base all'aereo. **Endpoint:** `http://localhost:3005/api/bookings`

- **AirlineRouteService** Gestisce le rotte aeree (collegamenti logici partenza-arrivo). Permette la ricerca di rotte esistenti e le operazioni amministrative di creazione e cancellazione. **Endpoint:** `http://localhost:3005/api/routes`
- **AirportsService** Gestisce l'anagrafica degli aeroporti. Espone metodi pubblici per la ricerca (codice, nome, città) e metodi protetti per l'inserimento di nuovi scali nel sistema. **Endpoint:** `http://localhost:3005/api/airports`
- **AirlineAirplaneService** Gestisce l'inventario degli aeromobili. Permette la ricerca per modello e la creazione di nuovi aerei definendo la capacità specifica per ogni classe (Economy, Business, First Class). **Endpoint:** `http://localhost:3005/api/airplanes`
- **AirlineStatService** Gestisce l'aggregazione dei dati per le statistiche. Recupera report su passeggeri totali, ricavi e performance delle rotte, filtrabili per data o email della compagnia. **Endpoint:** `http://localhost:3005/api/flights` (Path specifico: `/statistics`)

Workflow e uso dell'applicazione

Esperienza Utente e Prenotazioni

Un utente può effettuare ricerche di voli, incluse tratte con scali, e procedere alla prenotazione scegliendo posto a sedere, classe ed extra. Il volo prenotato viene visualizzato nella propria scheda personale; l'utente può decidere successivamente di annullare la prenotazione. In questo caso, la logica prevede che il posto a sedere venga effettivamente liberato per nuovi acquisti, mentre il biglietto rimane visibile nello storico dell'utente con lo stato aggiornato (soft delete), mantenendo traccia dell'operazione.

Gestione Compagnia Aerea (Airline)

Le compagnie aeree creano e gestiscono le proprie rotte in modo esclusivo. Questa scelta progettuale permette di intervenire su una specifica tratta (es. per interruzione del servizio), sospendendo o gestendo a cascata tutti i voli futuri associati a quella rotta. La compagnia può configurare nuovi aeromobili, definendo il layout dei posti per rispecchiare la realtà, e creare i singoli voli combinando le rotte e gli aerei precedentemente inseriti. Ogni volo può essere modificato (prezzi, orari) o cancellato logicamente ("soft delete"), con un aggiornamento automatico che si riflette su tutti gli utenti prenotati. Ogni compagnia ha inoltre accesso alle statistiche relative ai propri voli e ricavi.

Ruolo Amministratore (Admin)

L'amministratore ha il controllo completo sull'anagrafica: può inserire manualmente nuovi utenti o compagnie e cancellarli se necessario. È l'unica figura abilitata all'inserimento di nuovi aeroporti nel sistema e può aggiungere nuovi modelli di aeromobili. A livello di analisi, l'Admin ha il privilegio di visualizzare le statistiche globali di tutte le compagnie aeree.

Accesso e Profilo

Tutti gli utenti registrati hanno accesso a un'area profilo per modificare i propri dati personali e la password. Gli utenti non loggati (Guest) possono utilizzare il motore di ricerca per

visualizzare i voli disponibili e applicare filtri, ma il sistema impedisce loro di proseguire oltre la visualizzazione dei risultati

Avvio

in caso di problemi con le porte provare a riavviare winnat

```
net stop winnat  
net start winnat
```

per avviare mettersi nella cartella root del progetto e digitare

```
docker-compose up --build  
  
//per i logs  
docker logs taw-angular  
docker logs taw-server
```

- indirizzo server locale <http://localhost:4200/>

Dati di Seed

Utenti

Passeggeri

Campo	Mario Rossi	Luigi Verdi
email	mario@test.com	luigi@test.com
password	123456	123456
role	passenger	passenger
name	Mario	Luigi
surname	Rossi	Verdi
birthdate	1985-05-20	1990-10-10
paymentAddress	Milano	Roma
phonenumber	+393331234567	+393339876543

Admin

Campo	Valore
email	admin@bauflights.com
password	123456

Campo	Valore
role	admin
phonenumber	+3900000000000

Compagnie Aeree

Campo	BauAirlines	SwiftFly
email	info@bauairlines.com	info@swiftfly.com
password	123456	123456
role	airline	airline
company	BauAirlines	SwiftFly
phonenumber	+3902000000	+3906000000

Aeroporti

Code	Nome	Città	Paese	Ruolo nell'itinerario
LIN	Milano Linate	Milano	Italia	Partenza
FCO	Roma Fiumicino	Roma	Italia	Hub 1 (LuxAir)
MUC	Munich Airport	Monaco	Germania	Hub 2 (SwiftFly)
JFK	JFK International	New York	America	Destinazione finale

Aerei

Modello	Economy	Business	First Class
Airbus A350 Lux	20 righe × 6 posti (A-F) = 120 posti	5 righe × 4 posti (ACDF) = 20 posti	2 righe × 2 posti (AC) = 4 posti
Boeing 787 Swift	20 righe × 6 posti (A-F) = 120 posti	5 righe × 4 posti (ACDF) = 20 posti	2 righe × 2 posti (AC) = 4 posti

Rotte

#	Airline	Partenza	Arrivo
0	BauAirlines	LIN — Milano	FCO — Roma
1	BauAirlines	FCO — Roma	JFK — New York
2	SwiftFly	LIN — Milano	MUC — Monaco

#	Airline	Partenza	Arrivo
3	SwiftFly	MUC — Monaco	JFK — New York

Voli

Date Dinamiche Le date sono calcolate rispetto al momento in cui viene eseguito il seed.

+1 = domani, +2 = dopodomani

BauAirlines (Airbus A350 Lux)

#	flightNumber	Rotta	Giorno	Dep.	Arr.
0	LX100	LIN Milano → FCO Roma	+1	08:00	09:00
1	LX200	FCO Roma → JFK New York	+1	12:00	20:00
2	LX300	FCO Roma → JFK New York	+1	10:00	20:00

SwiftFly (Boeing 787 Swift)

#	flightNumber	Rotta	Giorno	Dep.	Arr.
3	SW500	LIN Milano → MUC Monaco	+2	10:00	12:00
4	SW600	MUC Monaco → JFK New York	+2	15:00	23:00
5	SW700	MUC Monaco → JFK New York	+2	13:00	23:00

Biglietti & Prenotazioni

Booking 1 — Mario, Diretto, Economy

Campo	Valore
Utente	Mario Rossi
Volo	LX100 — LIN Milano → FCO Roma
Classe	economy
Posto	10A

Booking 2 — Luigi, Diretto, Business

Campo	Valore
Utente	Luigi Verdi
Volo	LX300 — FCO Roma → JFK New York

Campo	Valore
Classe	business
Posto	1A

Booking 3 — Mario, Con Scalo, First Class

Campo	Tratta 1	Tratta 2
Volo	LX100 LIN → FCO	LX200 FCO → JFK
Posto	1C	2C
Classe	firstclass	firstclass

Booking 4 — Luigi, Con Scalo, Economy

Campo	Tratta 1	Tratta 2
Volo	LX300 FCO → JFK	SW500 LIN → MUC
Posto	15F	20D
Classe	economy	economy

Login



Bau Flights!

Bentornato a bordo!

[Non hai un account? Registrati qui](#)

Registrazione



Bau Flights!

Compagnia aerea per i tuoi amici a quattro zampe

Crea il tuo account !



Registrati

[Hai già un account ? Accedi](#)

Inserisci una email valida.

Azioni utente

Filtro voli

BauFlights

[Prenotazioni](#) [Profilo](#) [logout](#)

CITTÀ DI PARTENZA

Es. Milano

CITTÀ DI DESTINAZIONE

Es. New York

DATA DI PARTENZA

gg/mm/aaaa



COMPAGNIA (OPZIONALE)

Es. Alitalia

NUMERO VOLO (OPZIONALE)

Es. AZ203

DATA DI ARRIVO (OPZIONALE)

gg/mm/aaaa



CERCA VOLI

Risultati ricerca

Risultati: 5 soluzioni trovate

DIRETTO

COMPAGNIA :
BAU AIRLINES

NUMERO DI VOLO :
BAU100

CITTÀ DI PARTENZA
Milano
18/02 09:00
LIN

→

CITTÀ DI ARRIVO
Roma
18/02 10:30
FCO

PREZZO
€50.00

Prenota

DIRETTO

COMPAGNIA :
SKY WINGS

NUMERO DI VOLO :
SKY555

CITTÀ DI PARTENZA
Milano
18/02 11:00
LIN

→

CITTÀ DI ARRIVO
Roma
18/02 12:30
FCO

PREZZO
€55.00

Prenota

Scelta volo e posto a sedere con extra

BauFlights

Prenotazioni Profilo logout

COMPAGNIA AEREA
Bau Airlines

NUMERO DI VOLO
BAU100

ORARIO
09:00 - 10:30

ECONOMY - €50.00

BUSINESS - €150.00

FIRST CLASS - €300.00

Seleziona le opzioni extra

☐ POSTO EXTRA GAMBE - €15.00

☒ BAGAGLIO EXTRA - €30.00

☐ IMBARCO PRIORITARIO - €10.00

1A 1C 1D 1F

2A 2C 2D 2F

3A 3C 3D 3F

4A 4C 4D 4F

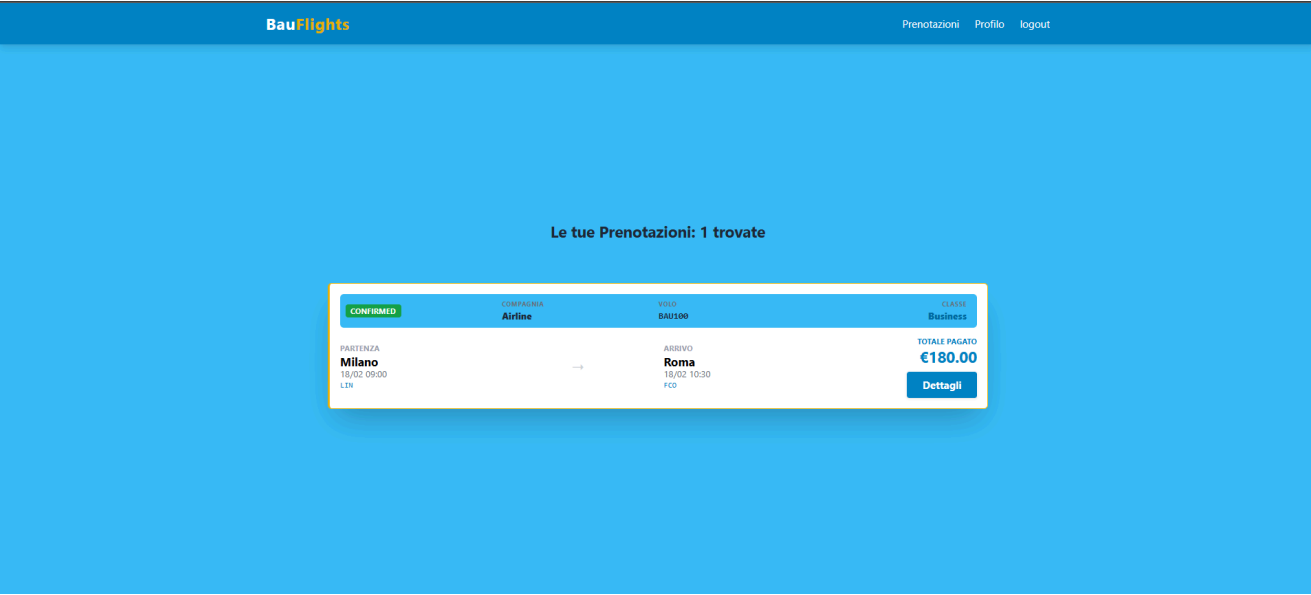
5A 5C 5D 5F

Prezzo andata: €180.00

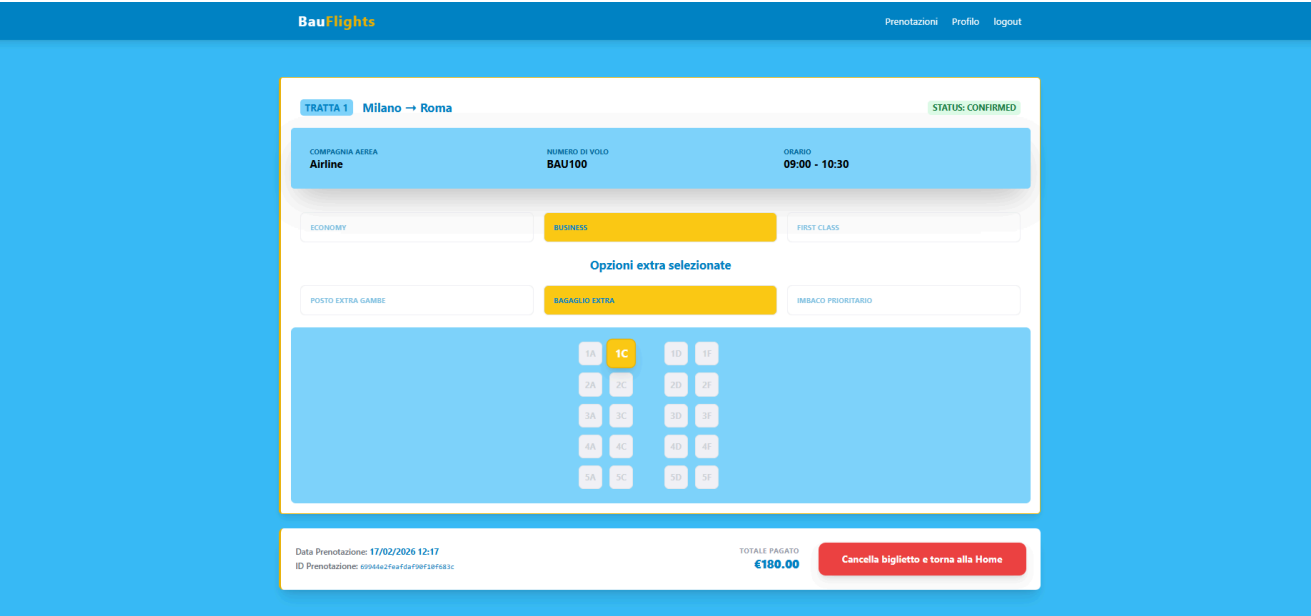
TOTALE DA PAGARE
€180.00

Conferma Prenotazione

Lista prenotazioni (link in alto sulla nav bar)



dettagli della prenotazione



Azioni compagnia aerea

Barra di controllo

Aggiunta Rotta e ricerca aeroporti

BauFlights

ProfiloLogout

Tool Creazione Rotte

CODICE AEROPORTO PARTENZA
MXP

CODICE AEROPORTO ARRIVO
VCE

CreaCerca

Tool ricerca Aeroporto

CODICE
Es. JFK

NOME
Es. Linate

CITTA'
Es. Milano

STATO
Italia

CERCA AEROPORTI

Aeroporti trovati con successo!

COD - LIN

Milano Linate
Milano
Italia

COD - MXP

Milano Malpensa
Milano
Italia

COD - FCO

Roma Fiumicino
Roma
Italia

Aggiunta veicolo

Aggiunta volo

Gestione Biglietti

BauFlights

ProfiloLogout

Gestione dei tuoi voli

Cerca Voli

CITTA' DI PARTENZA

Es. Milano

CITTA' DI DESTINAZIONE

Es. New York

NUMERO VOLO (OPZIONALE)

BAU100

DATA DI PARTENZA

gg/mm/aaaa

DATA DI ARRIVO (OPZIONALE)

gg/mm/aaaa

CERCA VOLI

VOLIO 1Milano → Roma

COMPAGNIA AEREA
Bau Airlines

NUMERO DI VOLO
BAU100

MODELLO AEREO
Boeing 737-800

PARTENZA

ARRIVO

18/01/2024

09:00

18/01/2024

10:30

Prezzi per classe

ECONOMY

BUSINESS

FIRST CLASS

50 €

150 €

300 €

Prezzi opzioni extra

POSTO EXTRA GAMME

BAGAGLIO EXTRA

INSICURO PRIORITARIO

15 €

30 €

10 €

ID Volo: amwacutaurba7m7drtfmcv

Salva ModificheElimina Volo

BauFlights

Profilo logout

Filtri Statistiche

DATA INIZIO
gg/mm/aaaa 📅
DATA FINE
gg/mm/aaaa 📅

Carica Statistiche

PASSEGGIERI TOTALI
25

RICAVO TOTALE
€ 7,945.00

Statistiche per Numero di Volo

NUMERO VOLO	N° VOLI	PASSEGGIERI TOT.	MEDIA PASS./VOLO	RICAVO TOTALE
BAU100	1	7	7.0	€ 750.00
BAU200	1	9	9.0	€ 5,270.00
BAU100	1	6	6.0	€ 935.00
BAU200	1	1	1.0	€ 990.00
BAU300	0	0	0.0	€ 0.00
BAU102	0	0	0.0	€ 0.00
BAU100	0	0	0.0	€ 0.00
AU12	0	0	0.0	€ 0.00
EYYY7	0	0	0.0	€ 0.00
eyyy	0	0	0.0	€ 0.00
EUUU	0	0	0.0	€ 0.00
AUUUU	0	0	0.0	€ 0.00

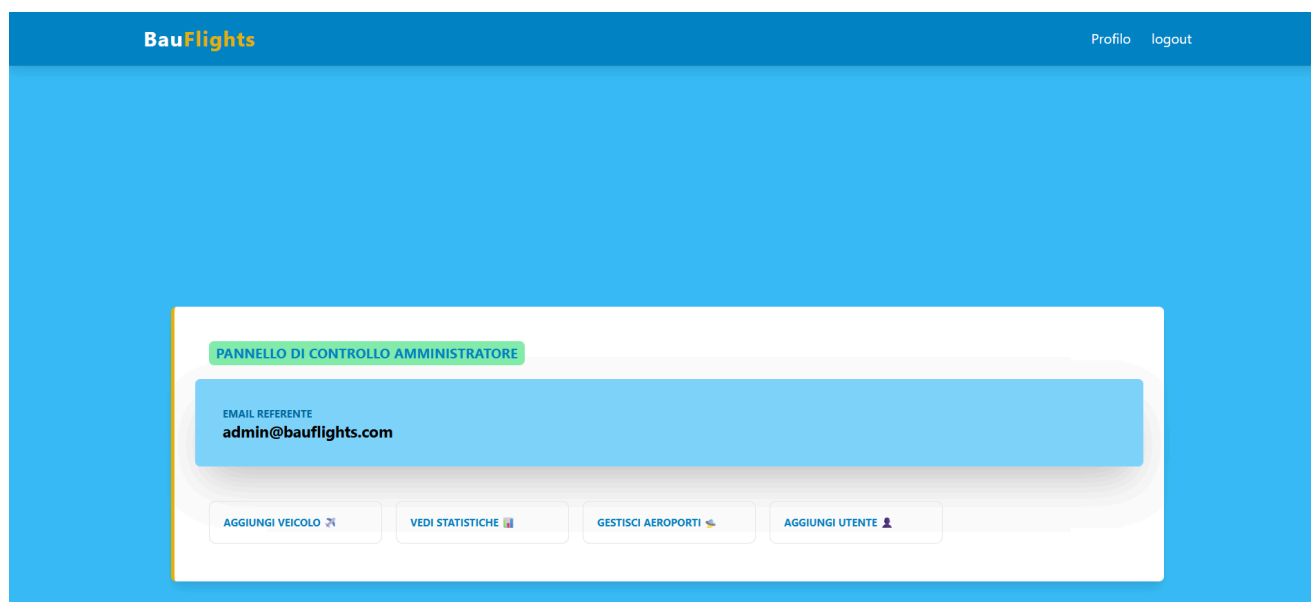
Statistiche per Rotta

LIN
Milano ➡️ FCO
Roma

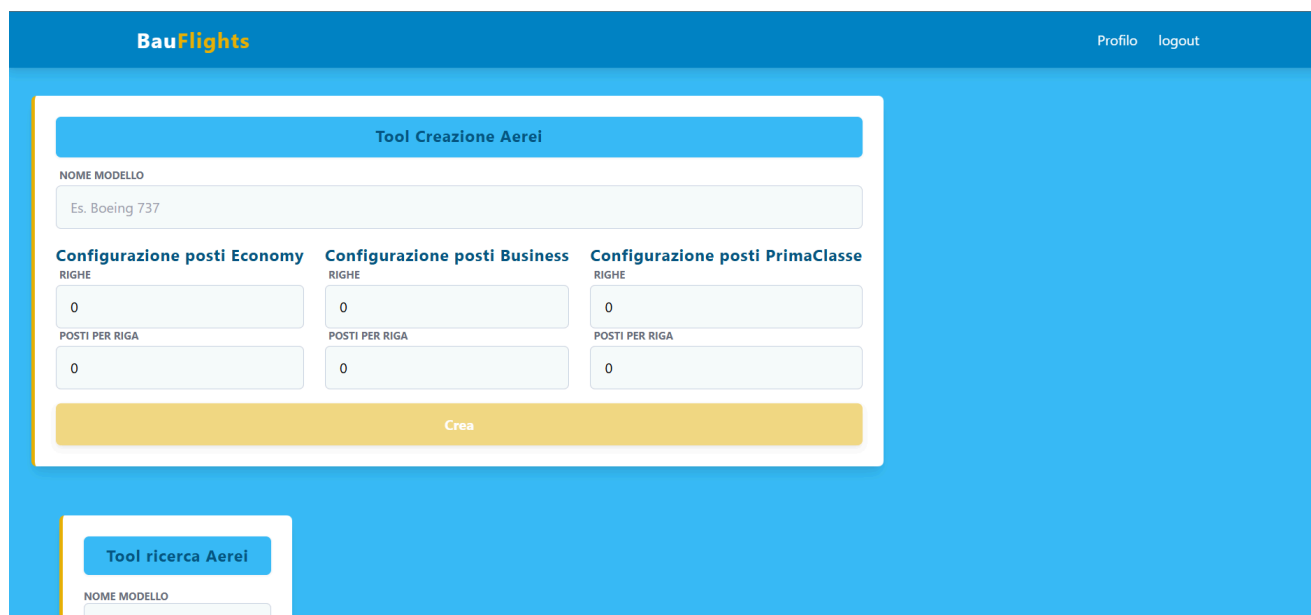
FCO
Roma ➡️ JFK
New York

Azioni Admin

pannello controllo



Aggiunta aerei



Aggiunta aeroporti

BauFlights

Profilologout

Tool Creazione Aeroporti

CODICE AEROPORTO

Es. MIL

NOME AEROPORTO

Es. Malpensa

CITTÀ

Es. Milano

PAESE

Es. Italia

Crea Aeroporto

Tool Ricerca Aeroporti

CODICE

Es. JFK

NOME

Es. LIN

CITTÀ

Es. Milano

PAESE

Es. Italia

Cerca Aeroporti

Statistiche admin

BauFlights

Profilologout

Filtri Statistiche

DATA INIZIO

gg/mm/aaaa

DATA FINE

gg/mm/aaaa

EMAIL COMPAGNIA (ADMIN)

info@bauairlines.com

Carica Statistiche

PASSEGGERI TOTALI

3

RICAVO TOTALE

€ 2,210.00

Statistiche per Numero di Volo

NUMERO VOLO	N° VOLI	PASSEGGERI TOT.	MEDIA PASS./VOLO	RICAVO TOTALE
LX100	1	2	2.0	€ 655.00
LX200	1	1	1.0	€ 1,555.00

Aggiunta e gestione Accounts

BauFlights

Profilo logout

Creazione Utenti

EMAIL

Es. esempio@mail.com

PASSWORD

Es. *****

ROLE

NOME

Es. Mario

COGNOME

Es. Rossi

DATA DI NASCITA

gg/mm/aaaa

PHONE NUMBER

Es. 1234567890

INDIRIZZO DI PAGAMENTO

Es. Mario

NOME COMPAGNIA

Es. Mario

Crea Utente

Tool Ricerca Account

EMAIL

mario

Cerca Account

Utenti trovati con successo!

EMAIL: mario@test.com

Mario Rossi

PASSENGER

Nata/o il: 20/05/1985

Tel: +393331234567

Indirizzo:

Milano

Elimina Utente

Profilo

BauFlights

Profilo logout

Informazioni del profilo

EMAIL

admin@bauflights.com

OLD PASSWORD

Es. *****

NEW PASSWORD

Es. *****

RUOLO

admin

Modifica Informazioni